ON-POLICY RESIDUAL LEARNING FOR DIFFUSION LANGUAGE MODELS

Yifan Yu[1,2,*], Yuqing Jian[1,*], Jisen Li[1,*], Yuezhou Hu[3], Tianyi Zhang[1], Xiaoxia Wu[1], Rui-Jie Zhu[4], Junxiong Wang[1], Zhongzhu Zhou[1,†], Fan Lai[2,†,‡], James Zou[1,5,†], Chenfeng Xu[1,6,†,‡]

[1]Together AI
[2]University of Illinois Urbana-Champaign
[3]University of California, Berkeley
[4]University of California, Santa Cruz
[5]Stanford University
[6]The University of Texas at Austin

[*]Equal contribution. [†]Equal advising. [‡]Corresponding authors.

ABSTRACT

Residual context (DiffusionGemma Team et al., 2026) is emerging as a key ingredient for unlocking the speed–quality Pareto frontier of diffusion language models (DLMs) by carrying forward predictive distributions across denoising steps. Yet turning this seemingly simple mechanism of computation recirculation into an effective and efficient learning paradigm is far from simple: there is no clear recipe for how residual context should be learned. We introduce ORL, a framework for on-policy residual learning in DLMs. Algorithmically, we identify that residual learning is inherently on-policy. ORL learns directly from these model-induced residual trajectories on the fly and introduces an entropy-contraction objective that encourages sharpening predictions as the model revisits its own intermediate predictions. System-wise, exact residual supervision requires capturing, retaining, and repeatedly processing many trajectory-dependent intermediate states. ORL co-designs the learning algorithm with asynchronous residual capture and transport, specialized block-diffusion attention, and packed compute-balanced training, making on-policy residual learning practical at scale. On a 4B DLM converted from Qwen3-4B, ORL improves tokens per forward (TPF) by up to 6.2× and, with our proposed DLMRT engine, end-to-end throughput by approximately 6.9× at comparable or better quality. In single-request serving on one H100, this model achieves a stronger speed–quality Pareto frontier than autoregressive models accelerated with EAGLE3 and DFlash speculative decoding under both strict and relaxed verification, running 1.74× faster than DFlash at matched accuracy. ORL also transfers to existing DLMs: on LLaDA-2.1-mini, 36 H100-hours and 4.2M self-generated tokens, about 10× less data than prior LLaDA recipes, yield up to 1.5× higher TPF and up to 3× higher throughput at comparable quality, and on DiffusionGemma, ORL raises TPF by 1.1–1.2×. With DLMRT and NVFP4 quantization, ORL models decode a single request at up to 5,046 tokens/s on one B200, 7,708 tokens/s on eight B200s, and 2,788 tokens/s on a consumer RTX 5090, with comparable quality.

Code | **Website** | **Models** | **Engine**

1 INTRODUCTION

Today's LLMs collapse the complexity of language generation into an extremely simple next-token prediction (NTP) paradigm. This simplicity delivers remarkable intelligence, but at a fundamental systems cost: autoregressive (AR) decoding remains sequential, severely underutilizing the massive parallelism of modern hardware (Nebius team, 2025; Frye & Biswas, 2026; Robbins, 2026;

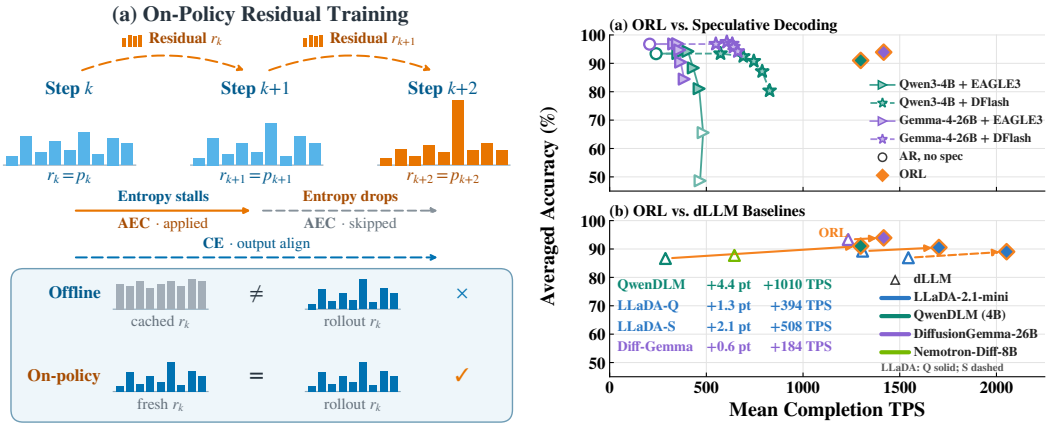


Figure 1: **(a)** On-policy residual training. Residual predictions r_k are carried across denoising steps. ORL trains on the fresh residuals produced by the current rollout (on-policy) rather than on cached residuals that no longer match the rollout (offline), and enforces entropy contraction across steps, applied only where entropy stalls, so that more tokens are committed. **(b)** A DLM target that learns to refine its residuals reaches the speculative-decoding frontier. Average GSM8K/HumanEval quality.

Killinger & Kiely, 2025). Unlocking parallel decoding for LLMs has emerged as one of the most timely problems in efficient language generation (Almeida, 2026; Liu et al., 2024; Yu et al., 2026; Bie et al., 2026). Among the most actively explored paradigms are discrete diffusion language models (DLMs). By initializing blocks of masked tokens and progressively unmasking them, DLMs transform each forward pass from a single-token prediction into an opportunity to decode many tokens (Nie et al., 2025; Arriola et al., 2025; Ye et al., 2025; Cheng et al., 2025).

Recently, there has also been an interesting debate: given speculative decoding, a lossless acceleration technique, why do we still need diffusion language models? We point out that DLMs open a different optimization dimension for language generation: parallelism that the model itself learns. By contrast, existing LLMs via next-token prediction optimize next-token accuracy. Speculative decoding (Chen et al., 2026a; Cheng et al., 2026; Huang et al., 2026) poses another dimension: it optimizes speed for a fixed target, and the attainable speedup is capped by how predictable the target already is. DLMs instead expose quality and parallelism as a joint learning problem: the model must learn not only what to predict but also how much reliable progress to make in each forward pass. Optimizing both dimensions in a single model is what makes a better speed–quality tradeoff possible than acceleration around a fixed target. It is one unique advantage of a DLM target (Figure 1b).

Yet optimizing the speed–quality tradeoff is difficult: beyond deciding which token should fill each position, the model must also determine how aggressively it should fill the block. Existing work tackles this challenge through improved diffusion objectives, decoding strategies, reinforcement learning, and iterative self-refinement (Hu et al., 2026b; DiffusionGemma Team et al., 2026; Lee et al., 2026; Wang et al., 2026b; Yu et al., 2026; Hu et al., 2026a; Ren et al., 2026). Among these directions, one idea is emerging as particularly natural and effective: recirculating residual predictions across decoding steps. In particular, each forward pass already spends substantial computation constructing a predictive distribution over the unresolved block. Rather than discarding this distribution and recomputing from scratch at the next step, residual decoding feeds it back as context, allowing subsequent steps to build upon the model's previous predictions (Hu et al., 2026b; Jo et al., 2026; Lee et al., 2026). Appendix B.1 shows that these intermediate predictions often anticipate the final tokens before they are committed. RCD (Hu et al., 2026b) and Looped Diffusion (Lee et al., 2026) show that this residual information can significantly improve the speed–quality Pareto frontier. DiffusionGemma (DiffusionGemma Team et al., 2026) pushes this trend even further, achieving over 20 tokens per forward (TPF; about 1,230 tokens/s on one H100, as shown in Figure 1b), significantly faster than existing speculative decoding.

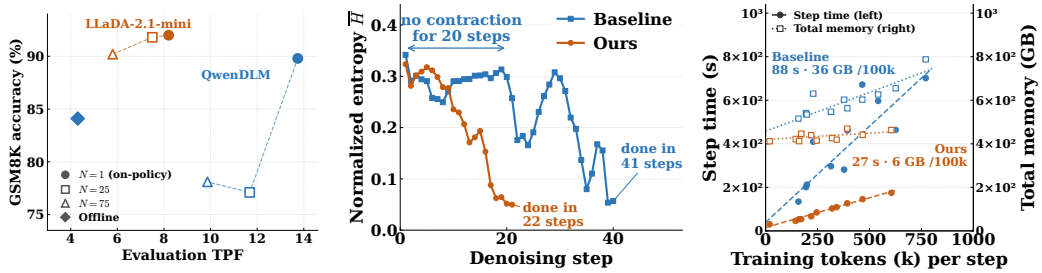
Existing methods, however, largely treat residual information as an architectural or decoding primitive, leaving no clear recipe for learning it effectively and efficiently. Algorithmically, residual learning is hard for two reasons. First, the learning loop is self-referential: Offline training can-

not faithfully reproduce the states seen at inference time, namely partially denoised blocks together with the residuals they receive. Even for on-policy training, under a shared optimizer, the residual pathway moves much faster than the backbone, and naïve updates destabilize training. Second, supervising intermediate states toward the final output indicates what each state should predict, but not how much progress each step should make. In practice, we find that entropy stays nearly flat across denoising steps, and TPF barely improves (Figure 2b). On the systems side, exact residual supervision requires generating intermediate predictions in flight and repeatedly feeding these states back to training. This trajectory-dependent workflow fits poorly into existing RL pipelines (§2).

We introduce ORL, an on-policy residual learning framework for DLMs that addresses each of these challenges. **(1)** Residual learning is inherently on-policy, and the loop it closes must be kept stable. ORL learns from residual trajectories generated online by the current model, using the exact residual inputs encountered during rollout (Figure 1a, bottom). It rescales the residual pathway so that it updates at the backbone's pace and gates supervision by rollout entropy, ensuring that already-confident predictions are not sharpened further. **(2)** Refinement speed needs its own objective. ORL introduces an adjacent entropy-contraction objective that explicitly requires each step to reduce the uncertainty left by the previous one, sharpening predictions along the residual trajectory (Figure 1a, top). On the systems side, **(3)** ORL makes exact on-policy residual learning practical at scale. It asynchronously captures residual states off the decoding critical path, uses memory-adaptive transport to keep or offload them depending on placement, and packs trajectory-dependent states into dense training batches. A compute-aware scheduler balances the resulting irregular workloads across GPUs, while a specialized staircase attention kernel executes packed block-diffusion states without materializing dense masks. Together, these designs make residual training efficient, while our proposed single-request engine, DLMRT, further converts fewer forward passes into higher inference throughput (Section 3.3).

Contributions. To the best of our knowledge, ORL represents the first open, end-to-end workflow for exact on-policy residual optimization in DLMs, and we release code, models, training data, the DLMRT inference engine, and a project page to facilitate reproduction and follow-up work. Our contributions are:

- **We identify residual refinement as an inherently on-policy learning problem.** Existing residual training suffers from two mismatches: offline residuals can induce trajectories different from those encountered at inference, while pre-generated trajectories become stale as the model evolves. This reveals that residuals must be optimized on the evolving state distribution they induce, with the residual pathway updated slowly enough that the loop stays stable.
- **We introduce a general recipe for on-policy residual optimization.** ORL trains intermediate states from fresh rollouts using their exact residual conditioning, distilling them toward verified final outputs, and explicitly improves step-wise refinement through adjacent entropy contraction with stabilized residual updates. The same recipe supports direct post-training of existing DLMs, without requiring a model-specific training pipeline.
- **We make exact trajectory-level residual optimization practical at scale.** We co-design the rollout and training system, including asynchronous in-engine trajectory capture, memory-adaptive residual transport, specialized block-diffusion attention, packed intermediate-state training, and compute-aware GPU balancing. These techniques speed up training by up to $5.2\times$.
- **We establish a stronger speed–quality frontier with little post-training.** With 60 H100-hours of post-training on QwenDLM, a 4B DLM converted from Qwen3-4B, ORL improves TPF by 2.5–$6.2\times$ and throughput by approximately $5\times$ at comparable quality (approximately $6.9\times$ together with DLMRT); in single-request serving on one H100, the regime speculative decoding targets, the resulting model is faster than every EAGLE3 (Li et al., 2025) and DFlash setting we measure: $1.74\times$ faster than DFlash at matched accuracy with relaxed verification, and $2.26\times$ faster than DFlash with strict verification, which keeps the AR's accuracy, 2.4 points above ours (Figure 1b). The same recipe improves LLaDA-2.1-mini by up to $1.5\times$ higher TPF and up to $3\times$ higher throughput, and DiffusionGemma by 1.1–$1.2\times$ higher TPF. ORL achieves these benefits with a few million self-generated tokens: on LLaDA-2.1-mini it uses 4.2M tokens and 36 H100-hours, about $10\times$ less data than the roughly 40M tokens of d3LLM-LLaDA and dParallel (Figure 7; Appendix H.3).
- **We design DLMRT, a low-latency single-request engine for block-diffusion DLMs.** Many practical setups, such as edge AI, serve one request at a time, where engines built for batching



(a) **Residual learning is on-policy.** (b) **Accuracy alone does not accelerate refinement.** (c) **Exact on-policy training is systems-heavy.**

Figure 2: **Three observations behind ORL. (a)** Only fresh trajectories jointly improve accuracy and TPF. **(b)** Standard residual supervision leaves entropy nearly flat across denoising steps. **(c)** Exact on-policy training incurs substantial time and memory overhead.

leave the GPU underused. Running each block's denoising loop on the GPU as one CUDA graph, DLMRT raises throughput by 1.2–1.6× over SGLang and vLLM without changing the decoding rule. With NVFP4 quantization, LLaDA-2.1-mini + ORL averages 5,046 tokens/s per single request across HumanEval on one B200, and DiffusionGemma + ORL averages 4,214 tokens/s across GSM8K on one B200 and 7,708 tokens/s with tensor parallelism across eight B200s. On a consumer GeForce RTX 5090, the two models reach 2,788 and 1,540 tokens/s on these benchmarks, respectively, matching or exceeding BF16 on an H100 (Section 4.3).

2 Related Work and Motivation

Residual diffusion decoding. A block DLM generates a response block by block: each block of B tokens is denoised over several steps, conditioned on the prompt and the previously committed blocks, which we call its *clean context*. At step k, the backbone f_θ observes a discrete canvas $s^{(k)} = (x^{(k)}, m^{(k)})$, where $x^{(k)} \in \mathcal{V}^B$ holds the current tokens and $m^{(k)} \in \{0,1\}^B$ marks the unresolved positions, which hold noise tokens (a mask token or random vocabulary tokens), and predicts a distribution $p_i^{(k)}$ over the vocabulary $\mathcal{V}$ for every position. A decoding rule then accepts the unresolved positions whose predictions are confident enough, for example under a probability threshold or an entropy budget, may edit accepted ones, and re-noises the rest; the block is committed once no position remains unresolved or a step cap is reached. The number of steps K of a block therefore depends on the model and the decoding rule, and TPF counts completion tokens per denoising forward. Residual decoding additionally carries the previous step's predictions forward as continuous context (Hu et al., 2026b; Jo et al., 2026; DiffusionGemma Team et al., 2026). Let $r_i^{(k)}$ denote the residual at position i and E_v the input embedding of token v. We inject its expected token embedding through a lightweight residual adapter g_ϕ with injection strength α:

$$\tilde{e}_i^{(k)} = E_{x_i^{(k)}} + \alpha\, g_\phi\big(z_i^{(k)}\big), \qquad z_i^{(k)} = \textstyle\sum_{v\in\mathcal{V}} r_{i,v}^{(k)} E_v, \qquad k > 1. \tag{1}$$

Here $r^{(1)} = 0$, and each position receives the distribution it produced at the previous step. For backbones with next-token (shifted) alignment, such as QwenDLM, the output at position i predicts position $i{+}1$, so $r_i^{(k+1)} = p_{i+1}^{(k)}$; same-position backbones, such as LLaDA, use $r_i^{(k+1)} = p_i^{(k)}$. The adapter is model-specific: QwenDLM uses a gated MLP warmed up offline, LLaDA a zero-initialized gated MLP added for ORL, and DiffusionGemma its native residual module (Appendix C). We call $\tau_{\theta,\phi} = \{(s^{(k)}, p^{(k)}, r^{(k)})\}_{k=1}^{K}$ a *residual trajectory*. We call the pair $(s^{(k)}, r^{(k)})$, a canvas together with the residual it receives, the *state* at step k, and index the states of a response by block and step, (b, k). The adapter and the residual it injects form the *residual pathway*.

Residual learning is inherently on-policy. Residual predictions directly affect which tokens are committed, which positions remain unresolved, and therefore which states the model encounters next. As the model changes, the residuals it produces and the trajectories induced by those residuals change together. This makes offline and stale residual supervision fundamentally mismatched. Offline residuals (Jo et al., 2026; Hu et al., 2026b), which are not taken from the current model's own

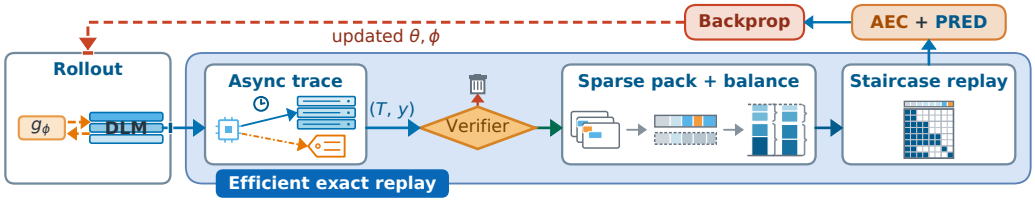


Figure 3: **Overview of ORL.** The current model generates residual-conditioned trajectories with exact state capture; verified states are packed and load-balanced across GPUs, replayed with staircase attention, and optimized using CE and AEC.

previous denoising step, may induce states different from those encountered by the model at inference time, while pre-generated trajectories (Ren et al., 2026) become stale as optimization updates the model. We define staleness N as optimizer steps between rollout-policy syncs ($N{=}1$ is on-policy; offline uses fixed cached trajectories). Existing on-policy DLM methods instead optimize token-space trajectories or final outcomes without directly learning the residual pathway (Wang et al., 2026b; Hu et al., 2026a). Figure 2(a) shows that offline training barely improves TPF, while increasing staleness sacrifices accuracy; only fresh trajectories improve both. Section 4.2 shows that training keeps changing the model's behavior, with rollout entropy decreasing steadily without collapse, so trajectories from earlier checkpoints become stale.

Accuracy supervision does not imply faster refinement. On-policy trajectories determine *where* residual learning should happen, but not *what behavior* it should encourage. Standard supervision trains each state toward the correct output, yet provides no pressure for one denoising step to make more progress than the previous one. In practice, we observe that the average normalized entropy of the unresolved positions (Section 3.1) remains nearly flat across many steps, even when the final answer is correct (Figure 2b). The model learns to predict accurately, but not to become progressively more decisive. For parallel decoding, this distinction is critical: improving final accuracy does not necessarily increase the number of tokens that can be committed per forward pass. Residual learning, therefore, needs an explicit objective for *refinement speed*, not accuracy alone.

Exact residual learning creates a new systems workload. Correct on-policy supervision requires training on the exact states seen during rollouts. These states are generated dynamically at every denoising step, must cross the rollout–training boundary without perturbing generation, and produce examples with highly irregular trajectory lengths and compute costs. This workload fits poorly into conventional RL training pipelines (Sheng et al., 2024; Fu et al., 2025). A naïve implementation repeatedly materializes these states, wastes memory on sparse trajectory structures, and suffers severe load imbalance. As Figure 2(c) and Appendix B.1 show, its time and memory costs grow rapidly. Thus, **on-policy residual learning requires a system around the states of residual trajectories.**

Appendix A discusses additional related work.

3 Method

Previous research discussed in Section 2 drew three main conclusions: residuals must be learned on the trajectories the current model induces (Figure 2a), refinement speed needs its own objective (Figure 2b), and exact on-policy supervision is a systems workload in its own right (Figure 2c). ORL answers all three within a loop (Figure 3). In each iteration, the model generates fresh residual-conditioned trajectories, a verifier keeps the successful rollouts, and ORL replays their states, re-evaluating each with the exact residual input seen during generation. Two objectives are applied: verified-output supervision teaches *what* each state should predict, and adjacent entropy contraction teaches *how quickly* predictions should sharpen across denoising steps. As every update changes the conditioning that generates the next batch, we control how fast the residual pathway moves (Section 3.1). Section 3.2 describes the training system that makes exact replay practical, and Section 3.3 the inference optimizations that turn fewer forward passes into higher throughput.

3.1 On-Policy Residual Learning

Because residuals and the states they induce change together with the model (Figure 2a), ORL regenerates its training data at every optimization step. At optimization step u, the model (θ_u, ϕ_u) generates G rollouts for each prompt c in minibatch $\mathcal{B}$. Each rollout $\mathcal{R}$ yields a final response $y^{\mathcal{R}}$ and its residual-conditioned trajectory. A verifier $V(c, y) \in \{0, 1\}$ keeps the successful rollouts $\mathcal{R}^+_{\mathcal{B}}$; rejected rollouts contribute no gradient. Each nonempty verified batch yields one optimizer step; its trajectories are then discarded, and the updated model generates the next batch. Within each kept rollout we train a subset $\mathcal{S}(\mathcal{R})$ of its states; for QwenDLM, for example, these are the 25% of states in each block that accept the most new tokens, plus the block's final state (Appendix E).

Equation 1 indicates that every step is conditioned on the previous step's prediction, so the canvas alone does not identify a state, and recovering the residual actually consumed would mean re-executing the earlier steps. ORL therefore preserves the *exact states* that the current model visited. For each selected state (b, k), we capture its canvas and the pre-adapter residual $z_i^{(b,k)}$ it consumed, computed with the rollout embedding table $E^{(u)}$. Replay evaluates Equation 1 on the captured canvas with $z_i^{(b,k)}$ held fixed by a stop-gradient, $\mathrm{sg}[z_i^{(b,k)}]$, so gradients reach the backbone and the adapter but not the embeddings through z. Replay places the captured canvas after its clean context and runs one forward pass under block-diffusion attention (Section 3.2), producing the replayed prediction $\widehat{p}_i^{(b,k)}$; model-specific normalization of the injected term, identical in rollout and replay, is omitted from Equation 1 (Appendix C). We store z rather than a vocabulary-sized distribution, and gradients update the backbone and the adapter at the captured state without backpropagating through earlier steps. Exactness here means preserving the captured input, not bitwise equality between rollout and replay outputs. Unlike experience replay in off-policy RL, each captured state is replayed once, by the parameters that generated it, and then discarded. Capturing these inputs without slowing generation is the first requirement of the training system in Section 3.2.

Learning what to predict. For every selected state of a verified rollout, we supervise the replayed prediction $\widehat{p}_i^{(b,k)}$ toward $y^{\mathcal{R}}_{b,i}$, the token at position i of block b in the verified response, with cross-entropy (CE), $\ell^{\mathrm{pred}}_{\mathcal{R},b,k,i} = -\log \widehat{p}^{(b,k)}_{i,\, y^{\mathcal{R}}_{b,i}}$, at both accepted and unresolved positions alike, so that an intermediate prediction corrected later learns directly from its eventual value. Token losses are averaged over the valid positions of each verified response and then across responses, giving $\mathcal{L}_{\mathrm{pred}}$, so that every verified response carries equal weight (Appendix C). Endpoint supervision alone, however, does not indicate how quickly a state should reach its prediction: a model can become more accurate while making little progress from one step to the next. This is the stall in Figure 2(b).

Learning how fast to refine. *Adjacent entropy contraction* (AEC) supplies the missing pressure by asking each step to reduce the uncertainty left by the previous one. Let $\mathcal{U}_{b,k} = \{i : m_i^{(b,k)} = 1\}$ be the unresolved positions, $\overline{H}(p) = -\sum_v p_v \log p_v / \log |\mathcal{V}|$ the normalized token entropy, and $H_{b,k} = \frac{1}{|\mathcal{U}_{b,k}|} \sum_{i \in \mathcal{U}_{b,k}} \overline{H}(p_i^{(b,k)})$ the average unresolved entropy of a state, computed from the detached rollout distribution for the source state ($H^{\mathrm{roll}}_{b,k}$, captured during generation) and from the replayed prediction $\widehat{p}$ for the next state ($H^{\theta,\phi}_{b,k+1}$). For a target contraction rate $\rho \in [0, 1]$,

$$\ell^{\mathrm{AEC}}_{\mathcal{R},b,k} = \Big[\mathrm{ReLU}\Big(H^{\theta,\phi}_{b,k+1} - (1-\rho)\ \mathrm{sg}\left[H^{\mathrm{roll}}_{b,k} \right] \Big) \Big]^2. \tag{2}$$

AEC rests on two design choices. First, it is a hinge: the loss is zero once the next state's unresolved entropy has dropped by at least the fraction ρ, and it penalizes only steps that fail to contract. A uniform entropy penalty would instead keep sharpening every prediction, including those that already make progress, at the risk of trading accuracy for confidence; the hinge acts only where refinement stalls (Figure 1a). Second, it targets the positions that remain unresolved after acceptance, since each entropy is taken over its own step's unresolved set. The only information the next step receives from the previous one is the tokens just accepted or edited and the residual carrying the previous step's predictive distribution. Thus, AEC rewards the model for turning that residual into confidence on the positions still to be decoded. Pair losses over $\mathcal{P}(\mathcal{R})$, the within-block adjacent pairs (b, k) whose next state $(b, k{+}1)$ is selected and whose two unresolved sets are nonempty, are averaged with equal weight per response into $\mathcal{L}_{\mathrm{AEC}}$ (Appendix C), and the full objective is $\mathcal{L}_{\mathrm{ORL}} = \mathcal{L}_{\mathrm{pred}} + \lambda_{\mathrm{AEC}} \mathcal{L}_{\mathrm{AEC}}$ with $\lambda_{\mathrm{AEC}} \geq 0$. This objective trains *what* each state predicts and

how efficiently the trajectory reaches it. As shown in Figure 2(b), AEC cuts decoding from 41 to 22 steps with the same answer.

Stabilizing the on-policy feedback loop. Because ORL trains on its own outputs, the loop can fail in two ways, and we guard against each. *The residual pathway can outrun the backbone.* Its outputs are the inputs of the next rollout, yet under a shared optimizer its relative update is about $38\times$ the backbone's, and QwenDLM's verifier reward collapses after roughly 250 training steps (Figure 8; Appendix H.2). We therefore keep the pathway's relative update size close to the backbone's: a pathway warmed up offline is rescaled by γ without changing its function, which under Adam shrinks its relative updates by about γ, and a pathway added from scratch starts from a zero output projection with a lower learning rate (Appendix C). *Supervision can reinforce what the model already believes.* Every CE target is a token the model generated itself, so at positions it already predicts confidently, CE only sharpens an existing belief, and it does so again at every iteration. We therefore apply CE only where the rollout is uncertain, with normalized entropy above a threshold ζ, and keep confident positions near their rollout prediction with a capped reverse KL.

3.2 Efficient Exact Residual Training

This loop is simple to describe but expensive to run. Exact replay turns each response into many states that must be captured during rollout, moved across the rollout–training boundary, and re-evaluated under block-diffusion visibility with highly irregular cost (Figure 2(c)). ORL removes these costs in the order data flows through the training loop (Figure 3), with three mechanisms that change how training executes, but not which states are supervised or which objective is optimized.

Asynchronous exact-state capture. The first cost is getting states out of the decode loop. Once generation advances, recovering an exact residual-conditioned input would require re-executing the preceding steps, while copying each state out synchronously would stall decoding. The engine instead snapshots the canvas, the pre-adapter residual z, and the rollout statistics the objectives need (the source entropies for AEC and, when the entropy gate is used, the output-head inputs from which its reference is rebuilt) at the moment they are produced, enqueues GPU-side handles, and transfers them off the decode critical path. When rollout and training are colocated, the block-local buffers holding z, which we call residual pages, stay GPU-resident and are handed off through CUDA-IPC rather than round-tripping through host memory (Appendix D.1).

Packed and cost-balanced state training. Captured states form the irregular workload of Section 2. Each verified rollout contributes a variable number of states and AEC pairs with different context lengths and residual footprints, so evaluating them one by one recomputes shared clean context and leaves GPUs idle. Following the slicing of TraceRL (Wang et al., 2026b), a row holds a response's clean tokens once together with the states of its blocks at the same step k, so each state reuses the clean context before it. We pack rows from different responses as independent documents, keep residuals as sparse residual pages rather than full-sequence tensors, and assign rows to data-parallel groups in order of largest estimated cost first. Each group returns the numerators and denominators of the response-normalized losses, which are summed globally before division, so packing and redistribution leave the objective unchanged up to floating-point rounding (Appendix D.3).

Staircase attention kernel. Packing leaves one obstacle: each packed row mixes documents, clean context, and noisy blocks, and replay must still respect block-diffusion visibility. A noisy block attends to the prompt, the preceding clean blocks, and itself, but never to its own clean copy, which is present in the row as context for later blocks, to later blocks, or to other noisy blocks; clean queries never attend to noisy keys. A dense mask costs $O(N^2)$ storage and dispatches fully invisible tiles. Our tiled Triton kernel encodes the rule implicitly from per-document metadata, skips invisible tiles, and supports execution over packed variable-length documents with grouped-query attention (Appendix D.2). Figure 5 measures the cumulative effect of the three mechanisms.

3.3 Efficient Residual Inference

Fewer forward passes do not automatically mean faster decoding. Residual conditioning adds work to every forward pass, and once a request needs fewer denoising steps, costs that do not shrink with the step count, such as prefill and per-block cache writes, take a larger share of its time (Appendix F). We therefore optimize the decoder without changing its decoding rule. Within each model family,

Table 1: **Generation quality and parallel decoding efficiency.** We report benchmark scores (%) and tokens per forward (TPF). Our results evaluate a single checkpoint across all benchmarks. Base and ORL use their selected family-specific decoders (Appendix E.1), so arrows compare complete model–decoder systems rather than weights alone. Within each model-family block, the best result is **bolded** and the starting point for ORL is underlined. Red arrows indicate ORL's TPF improvement.

Model / training	GSM8K		MATH-500		HumanEval		MBPP	
	Acc.↑	TPF↑	Acc.↑	TPF↑	pass@1↑	TPF↑	pass@1↑	TPF↑
Qwen-based models and other DLM baselines								
d3LLM–Dream	81.4	4.9	38.2	3.9	57.1	3.2	55.6	3.0
Fast-dLLM-v2-7B	77.5	2.2	48.7	2.6	61.7	2.6	50.1	2.0
RCD–SDAR-4B-b64	87.0	—	68.4	—	—	—	—	—
TraDo-4B (TraceRL, dynamic)	90.3	—	71.8	—	—	—	—	—
SDAR-8B-b32	88.9	2.9	63.6	4.8	73.5	2.4	58.0	2.4
QwenDLM (4B)	90.4	2.4	88.0	2.1	82.9	1.8	79.2	2.1
+ ORL	**92.4**	**5.9** ↑+3.5	**92.2**	**5.8** ↑+3.7	**89.6**	**11.1** ↑+9.3	**79.4**	**8.0** ↑+5.9
LLaDA-based models								
Fast-dLLM–LLaDA	74.7	2.8	30.8	2.0	37.8	2.6	38.6	2.1
dParallel–LLaDA	72.6	5.1	30.2	3.2	39.0	4.9	40.0	2.4
D2F–LLaDA	73.2	2.9	28.7	2.4	36.6	2.7	38.0	1.9
d3LLM–LLaDA	73.1	**9.1**	30.4	5.7	39.6	6.0	40.6	4.2
dOPSD–LLaDA-8B-Instruct	72.87	—	36.00	—	39.63	—	39.54	—
dUltra-coding	81.5	8.4	35.6	6.7	35.9	7.2	37.0	7.3
DMax-Math	**92.1**	5.5	75.4	5.9	—	—	—	—
LLaDA-2.1-mini	91.9	5.6	84.6	8.2	86.6	11.3	**77.5**	7.7
+ ORL	92.0	8.5 ↑+2.9	**87.0**	**10.2** ↑+2.0	**89.0**	**13.1** ↑+1.8	77.3	**10.3** ↑+2.6
DiffusionGemma-based models								
DiffusionGemma	95.8	26.0	**92.3**	23.7	90.9	24.5	**87.7**	23.1
+ ORL	**96.0**	**30.0** ↑+4.0	**92.3**	**27.4** ↑+3.7	**91.9**	**27.9** ↑+3.4	87.4	**28.7** ↑+5.6

Base and ORL run on the same engine, so throughput comparisons between them reflect training rather than engine differences.

Commit folding. After a block is finished, some existing decoders (e.g., the LLaDA-2.1-mini decoder) run one more forward pass on the clean block to check that no token should still be edited and to write the block's key–value cache. Following the inference engine of the LLaDA-2.1 team (Bie et al., 2026; Ant Group DeepXPU Team & SGLang Team, 2025), we fold this pass into the next block's first denoising step, so a single forward pass processes the clean previous block and the masked current block together. If the check edits a token, the current block's half is discarded and the joint pass is repeated, so every block still sees the context it would see without folding. We extend this folding to more models, including QwenDLM and the residual-conditioned ORL models. Saving one forward pass per block gives the largest single gain, 53% higher throughput (Figure 11). CUDA graphs for prompt prefill and the residual adapter, fused acceptance kernels, metadata reuse, and gradient-free scheduling remove most of the remaining host overhead; together with commit folding, they raise the HumanEval throughput of LLaDA-2.1-mini with ORL by 88%. DLMRT applies the same folding.

A single-request engine. General-purpose serving engines are designed around batching many requests. However, many practical setups require efficiently serving a single request, such as edge AI on local or consumer GPUs. Single-request decoding is also where parallel decoding pays off most: at batch size 1, decoding is memory-bound and most GPU compute sits idle, which is exactly the compute a DLM spends on committing many tokens per forward; it is also the regime speculative decoding targets. Each such forward pass is small, however, so kernel launches and scheduling take a large share of its time. We therefore develop DLMRT, an inference engine dedicated to single-request diffusion decoding for all three model families: it runs each block's denoising loop on the GPU as one CUDA graph and rewrites the decoding kernels around a single request (Appendix F.1). We release DLMRT as a separate repository, and Section 4.3 compares it with state-of-the-art engines.

4 EXPERIMENTS

Evaluation Methodology. We evaluate ORL from two popular starting points on nine benchmarks. For *AR start*, we convert Qwen3-4B-Thinking-2507 (Yang et al., 2025) into a residual-conditioned DLM, termed QwenDLM, through an SDAR-style offline training pipeline, and continue optimization with ORL. For *DLM start*, we post-train LLaDA-2.1-mini and DiffusionGemma (DiffusionGemma Team et al., 2026). Full training and evaluation details are in Appendix E.

4.1 MAIN RESULTS

ORL substantially improves both quality and parallelism. Table 1 compares ORL with prior DLM baselines, including TraceRL's TraDo-4B (Wang et al., 2026b) and the on-policy self-distillation method dOPSD (Dat et al., 2026).[1] From QwenDLM, it improves TPF by 2.5–6.2× while preserving or improving quality. On HumanEval, for example, ORL reaches 89.6 pass@1 at 11.1 TPF, compared with 73.5/2.4 for SDAR-8B. The same advantage transfers to existing DLMs: on both LLaDA-2.1-mini and DiffusionGemma, ORL raises TPF on all four benchmarks while maintaining or improving quality. For example, on HumanEval, ORL improves LLaDA-2.1-mini from 86.6/11.3 to 89.0/13.1 in pass@1/TPF. On GSM8K and MATH-500, it reaches 92.0/8.5 and 87.0/10.2 in accuracy/TPF, compared with 92.1/5.5 and 75.4/5.9 for DMax-Math (Chen et al., 2026b). Appendix G compares additional methods that are not included in Table 1 because they lack public checkpoints or cannot be evaluated under aligned settings.

ORL generalizes across models and domains. Benchmarks in Table 1 cover standard math and code evaluations but do not establish whether the same gains transfer broadly. Table 2 evaluates direct post-training of LLaDA-2.1-mini and DiffusionGemma on five additional benchmarks spanning mathematics, knowledge-intensive reasoning, code generation, and instruction following. Across two independently developed DLMs, ORL raises TPF on all ten model–benchmark pairs (1.26× on average, up to 1.46×), with little quality change.

Table 2: **Generalization across domains.** Post-training of LLaDA-2.1-mini and DiffusionGemma (DG) across additional benchmarks. Each entry reports *quality / TPF*.

Dataset	LLaDA Base	LLaDA +ORL	DG Base	DG +ORL
AIME-24	**50.0**/5.31	48.3/**7.77**	55.8/18.15	**56.3/22.02**
MMLU-Pro	57.7/3.60	**58.2/4.88**	**78.6**/18.57	78.1/**22.60**
GPQA-D	**49.5**/3.05	46.1/**4.21**	**65.7**/17.50	65.3/**21.27**
LCB-v6	28.0/5.90	**30.3/7.43**	**53.5**/19.96	51.4/**24.09**
IFEval	**84.1**/1.53	82.3/**1.74**	**66.9**/9.90	**66.9/11.68**

ORL improves the quality–throughput frontier. Because TPF ignores the per-forward cost of residual conditioning and block-diffusion execution, we also measure output throughput for a single request on one H100, running the DLMs on DLMRT (Figure 1(b)). QwenDLM trained with ORL reaches 1,298 tok/s at 91.0% average GSM8K/HumanEval accuracy, about 6.9× faster than its starting DLM running on SGLang (187 tok/s), and about 5× when ORL also runs on SGLang (Appendix G.1). AR decoding reaches 240 tok/s. Among measured speculative settings with mean accuracy at least ORL's minus 0.5 percentage points, the fastest EAGLE3 (Li et al., 2025) and DFlash settings reach 405 and 746 tok/s using relaxed verifier thresholds (τ=0.1 and 0.03); with strict verification (τ=1), they reach 353 and 573 tok/s. These are system-level comparisons using each method's native decoder and clean-request throughput, with AR and speculative decoding running on vLLM. Appendix G reports the full accuracy, failure-count, and verification-threshold context. Sweeping the decoding threshold further shows that ORL shifts the whole quality–throughput frontier outward rather than selecting a more aggressive threshold, improving clean throughput by 5.60× on QwenDLM and 2.95× on LLaDA-2.1-mini at the highlighted comparable-accuracy pairs (Figure 4; Appendix G.4). Appendix J characterizes these accuracy–TPF curves. Unlike speculative decoding, whose speedup is capped by how predictable a frozen target already is, ORL optimizes both speed and quality, with advantage widening with block size (Figure 6).

Rollout freshness changes learning dynamics. Our multi-seed study compares every-step rollout refresh ($N = 1$) with infrequent refresh ($N = 25, 75$), using the same initialization, verifier, and optimization objective (Figure 2a). All settings generate new trajectories; N controls rollout-weight refresh. After convergence, the fresh-rollout setting reaches 89.8% mean GSM8K accuracy and 13.7 TPF, versus 77.1%/11.7 and 78.1%/9.9 for $N = 25$ and $N = 75$, respectively. Fresh rollouts start

[1] Baseline results are quoted from published papers; the MATH-500 entries of rows from the d3LLM evaluation are MATH (4-shot) on the full test set. Appendix E.1 lists the source of each row.

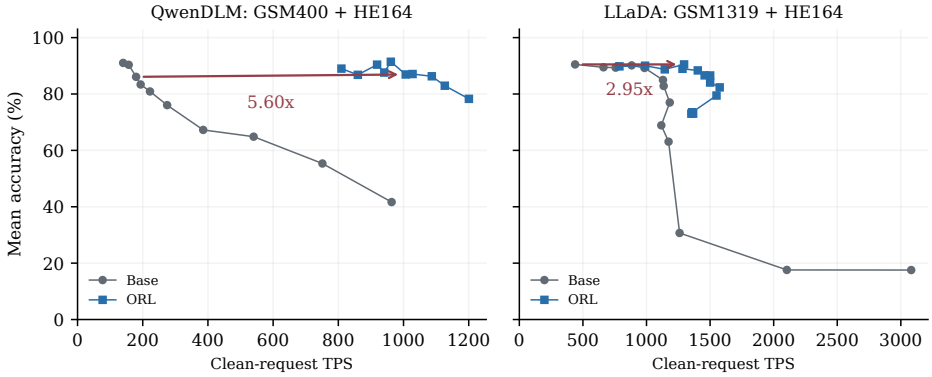


Figure 4: Common-task quality–throughput sweeps: mean GSM8K/HumanEval accuracy versus clean-request TPS for every Base and ORL setting. Arrows mark the highlighted comparable-accuracy pairs, with 5.60× (QwenDLM) and 2.95× (LLaDA-2.1-mini) clean-TPS gains. Appendix G.4 lists the settings and reports completed-output TPS. All settings are measured with SGLang.

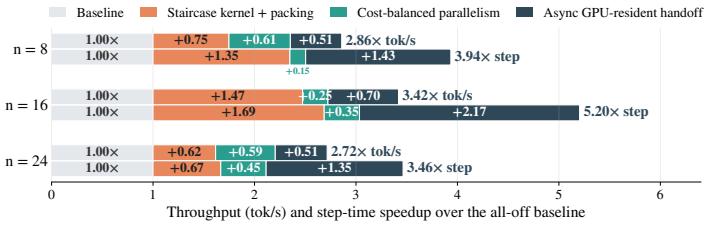


Figure 5: **Cumulative system optimizations.** End-of-bar values report system speedups in training-step time and token throughput.

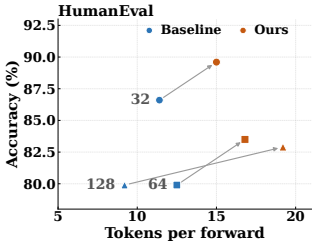


Figure 6: **Scaling with training block size.**

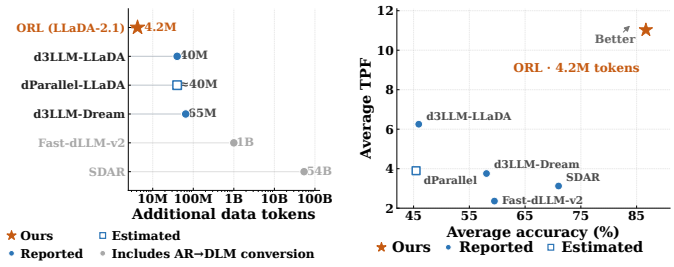
lower but overtake the infrequent-refresh variants. These results motivate frequent refresh for the evaluated recipe.

4.2 Ablations and Analysis

System co-design makes exact on-policy training practical. Figure 5 cumulatively enables staircase attention and packed state training, compute-aware balancing, and asynchronous GPU-resident handoff. Across $n = 8/16/24$ rollouts, the complete system reduces optimizer-step time by $3.9/5.2/3.5\times$ and raises token throughput by $2.9/3.4/2.7\times$ over the all-off implementation. Detailed phase breakdowns are provided in Appendix H.1.

ORL is highly data-efficient. Figure 7(a) compares the amount of additional training data used, and Figure 7(b) the quality–TPF operating point it reaches. On LLaDA-2.1-mini, ORL consumes 4.2M tokens, the prompts and responses of the 5,000 rollouts it generates over 625 training steps (one prompt and eight rollouts per training step), about $10\times$ less than the roughly 40M tokens of dParallel and d3LLM-LLaDA. ORL still reaches a higher average TPF at higher average accuracy than every LLaDA-, Dream-, and Qwen-based baseline. The run takes only 4.5 hours on a single node with eight H100 GPUs (36 H100 GPU-hours), and training ORL on QwenDLM takes 60 H100 GPU-hours. Appendix H.3 details how token counts and GPU-hours are measured.

Residual scaling stabilizes training without sacrificing optimization progress. Over the first 250 training steps, verifier reward is nearly identical with and without scaling. The difference emerges over longer training. With scaling, reward remains high at 0.677, whereas the unscaled run collapses to 0.225 (Figure 8). Removing scaling changes the backbone update magnitude by only $0.96\times$, but increases residual-adapter updates by $31.3\times$ (Appendix H.2).



(a) Data efficiency (b) Performance comparison

Figure 7: **(a–b)** With only a small amount of self-generated on-policy training data, ORL rapidly improves refinement efficiency and achieves a stronger quality–TPF tradeoff.

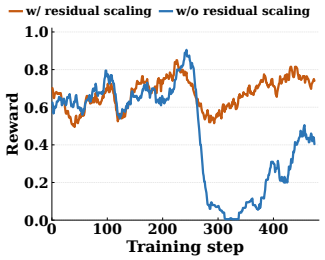


Figure 8: **Residual scaling preserves stable long-horizon optimization.** QwenDLM trained with ORL on GSM8K prompts.

Table 3: **Each component matters, and performance is robust to hyperparameters.** Accuracy (%) / TPF on GSM8K and MATH-500. All variants train QwenDLM with ORL on GSM8K.

(a) Training-design ablation			(b) Hyperparameter sensitivity		
Configuration	GSM8K	MATH-500	Configuration	GSM8K	MATH-500
Full	**94.5 / 6.25**	**90.0 / 5.64**	**Default**	**94.5 / 6.25**	**90.0 / 5.64**
− AEC	92.6 / 4.48	92.6 / 4.38	$\lambda_{\text{AEC}} = 12.5$	94.6 / 5.53	92.8 / 5.06
− entropy-gated RKL	87.8 / 3.29	76.8 / 3.46	$\lambda_{\text{AEC}} = 200$	92.6 / 8.53	88.2 / 7.42
− residual pathway	92.0 / 3.68	75.0 / 3.56	RKL cap $= 0.1$	93.6 / 6.51	92.2 / 5.83
			RKL cap $= 20$	93.8 / 6.80	93.2 / 5.89

ORL scales with available refinement capacity. Figure 6 varies the training block size of LLaDA-2.1-mini from 32 to 128. On HumanEval, TPF improves from 11.4 to 15.0 at block size 32 and from 9.2 to 19.2 at block size 128. MBPP shows the same trend (Appendix H.2).

Effective and robust training. Table 3 reports the contribution of each training component and the sensitivity to the main optimization hyperparameters. Removing AEC cuts TPF by 28% on GSM8K and 22% on MATH-500 while changing accuracy by -1.9 and $+2.6$ points, so AEC mainly buys speed, and λ_{AEC} sets this trade-off (Table 3(b)); removing entropy-gated reverse KL or the residual pathway degrades both accuracy and TPF, costing up to 15 points on MATH-500. AEC also gives a better accuracy–TPF tradeoff than a naive entropy loss at different weights (Table 4). In contrast, ORL is robust to its hyperparameters: lowering the AEC weight by $4\times$ or varying the RKL cap over a $200\times$ range lowers accuracy by at most 0.9 points and mainly shifts TPF.

How on-policy training changes model behavior. Figure 9 (orange) tracks the entropy of the rollout policy over training: it decreases steadily from 0.20 to 0.14 rather than collapsing, matching the graded increase in TPF. Figure 9 (blue) tracks the covariance between the log-probability of sampled tokens and their advantage-weighted probability (Appendix I.3), the quantity that sets the direction of entropy change under a policy-gradient update. It stays positive on average, so successive updates sharpen the distribution, and it shrinks toward zero as training proceeds, so sharpening slows down instead of running away (Cui et al., 2025; Jin et al., 2026). Together with the residual-to-embedding norm ratio, which rises from 0.91 to 1.08 during on-policy training (Figure 19), and the bounded residual-pathway update ratios in Appendix H.2, this indicates that ORL changes how intermediate predictions are refined across denoising steps rather than merely improving the final-token predictor, and that it does so by making greater use of residual context without destabilizing the residual pathway.

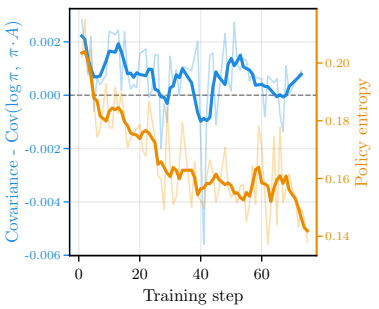


Figure 9: Dynamics of on-policy residual learning. The entropy-change covariance (blue, left axis) remains positive but approaches zero, while rollout entropy (orange, right axis) decreases gradually without collapse.

Table 5: Clean-request throughput (tokens/s) of the prior engines and DLMRT for a single request on one H100, with the same checkpoints and decoding settings; red arrows give the speedup of DLMRT over the prior engine. The prior engine is SGLang for QwenDLM and LLaDA-2.1-mini and vLLM for DiffusionGemma.

Model		GSM8K		HumanEval	
		Prior	DLMRT	Prior	DLMRT
QwenDLM	Base	210	328 ↑1.56×	164	249 ↑1.52×
	+ ORL	718	987 ↑1.37×	1148	1609 ↑1.40×
LLaDA-2.1-mini	Base	716	918 ↑1.28×	1302	1698 ↑1.30×
	+ ORL	949	1281 ↑1.35×	1530	2124 ↑1.39×
DiffusionGemma	Base	990	1271 ↑1.28×	963	1196 ↑1.24×
	+ ORL	1132	1456 ↑1.29×	1128	1378 ↑1.22×

Table 6: ORL models in BF16 on one H100 and with NVFP4 quantization on one B200, all throughput measured with DLMRT. Each cell reports accuracy (%) / TPS; red arrows give the speedup of NVFP4 on B200 over BF16 on H100, which combines the newer GPU with the lower precision. BF16 accuracy is that of Table 1 and BF16 TPS that of DLMRT in Table 5. NVFP4 (NVIDIA) is NVIDIA's NVFP4 checkpoint of the DiffusionGemma base model (NVIDIA, 2026); NVFP4, TP8 runs DiffusionGemma + ORL with tensor parallelism across eight B200s (generation seed 1234). Appendix F.2 adds TPF.

Model	Precision	GSM8K	MATH-500	HumanEval	MBPP
LLaDA-2.1-mini					
+ ORL	BF16	92.0 / 1281	87.0 / 1627	89.0 / 2124	77.3 / 1539
+ ORL	NVFP4	92.0 / 2953 ↑2.31×	84.0 / 3882 ↑2.39×	89.0 / 5046 ↑2.38×	75.9 / 3564 ↑2.32×
DiffusionGemma					
+ ORL	BF16	96.0 / 1456	92.3 / 1415	91.9 / 1378	87.4 / 1361
Base	NVFP4 (NVIDIA)	96.1 / 3627	92.7 / 3198	91.3 / 3464	87.1 / 2991
+ ORL	NVFP4	95.6 / 4214 ↑2.89×	93.4 / 3805 ↑2.69×	91.9 / 3949 ↑2.87×	86.3 / 3898 ↑2.86×
+ ORL	NVFP4, TP8	96.1 / 7708 ↑5.29×	91.4 / 7123 ↑5.03×	92.7 / 7026 ↑5.10×	86.0 / 6871 ↑5.05×

Table 7: Throughput (tokens/s) of the ORL models with NVFP4 quantization on one consumer GeForce RTX 5090, measured with DLMRT (DiffusionGemma: generation seed 1234).

Model	GSM8K	MATH-500	HumanEval	MBPP
LLaDA-2.1-mini + ORL	1648	2081	2788	2053
DiffusionGemma + ORL	1540	1408	1490	1465

AEC versus naive entropy minimization. Table 4 compares AEC with a naive entropy loss that uniformly minimizes prediction entropy with weight λ. A small weight (λ=4) lowers both accuracy and TPF relative to AEC, while a large weight (λ=50) raises TPF only from 7.7 to 8.2 at a 14.8-point accuracy cost. Because AEC penalizes only steps whose unresolved entropy fails to contract, it reaches a better accuracy–TPF tradeoff.

Table 4: AEC versus a naive entropy loss on QwenDLM: mean GSM8K accuracy (%) / TPF over independent training runs and repeated evaluations. This ablation trains only on math prompts because of compute limits.

Objective	GSM8K
AEC (λ_{AEC}=50)	**91.6** / 7.7
Naive entropy loss (λ=4)	90.0 / 7.1
Naive entropy loss (λ=50)	76.8 / **8.2**

4.3 Deployment Efficiency

A dedicated engine. Table 5 compares DLMRT (Section 3.3) with the prior engines on the same checkpoints and decoding settings, under the measurement protocol of Appendix E.1. Across the

measured pairs, DLMRT raises throughput by 1.24–1.56× for the Base models and by 1.22–1.40× for the ORL models. For QwenDLM and LLaDA-2.1-mini, these measurements use a DLMRT mode that reproduces SGLang's forward pass and decoding decisions (4,816 of 4,820 QwenDLM answers are identical to SGLang's); for DiffusionGemma, DLMRT follows the reference implementation's BF16 roundings. Across all three families, accuracy under DLMRT stays within 0.6 points of Table 1 (Appendix F.1).

Combining ORL with NVFP4 quantization. ORL reduces the number of forward passes, while low-precision inference makes each forward pass cheaper, so the two can be combined. We quantize the routed experts of the ORL-trained LLaDA-2.1-mini and DiffusionGemma to NVFP4, with 4-bit weights and activations, keep attention, routers, and shared experts in BF16, and use an FP8 KV cache. Each 16-weight block takes the FP8 scale that minimizes its layer-output error under a local Hessian of the layer's input activations (NVIDIA Model Optimizer Team, 2026); for DiffusionGemma, only the diagonal of this Hessian is used. Because only the scales change, the quantized checkpoints run on standard NVFP4 kernels without extra overhead (Appendix F.2).

NVFP4 gives large speedups with small accuracy changes: on one B200, it raises throughput over BF16 on one H100 by 2.31–2.39× for LLaDA-2.1-mini and 2.69–2.89× for DiffusionGemma (Table 6). With ORL and NVFP4, LLaDA-2.1-mini reaches 5,046 tokens/s on HumanEval. DiffusionGemma + ORL in NVFP4 is also 1.14–1.30× faster than NVIDIA's NVFP4 release of DiffusionGemma (NVIDIA, 2026) at comparable accuracy (within 0.8 points; Table 6), measured back to back on one B200 with DLMRT. With tensor parallelism across eight B200s, DiffusionGemma + ORL reaches 7,708 tokens/s on GSM8K. Fast decoding does not require datacenter GPUs: the NVFP4 ORL models fit in the 32 GB of a single consumer GeForce RTX 5090, where DLMRT runs LLaDA-2.1-mini at 2,788 tokens/s on HumanEval and DiffusionGemma at 1,540 tokens/s on GSM8K (Table 7), matching or exceeding BF16 on one H100 on every benchmark (Table 6).

5 Conclusion

We introduced ORL, a framework for on-policy residual learning in DLMs. ORL trains on the verified residual-conditioned trajectories that the current model induces, adds an entropy-contraction objective that sharpens predictions across denoising steps, and co-designs the training system so that exact on-policy supervision is practical. From QwenDLM, our 4B AR-converted DLM, ORL improves TPF by 2.5–6.2× and throughput by approximately 5× at comparable quality (approximately 6.9× together with DLMRT), above the measured EAGLE3 and DFlash operating points. Applied to LLaDA-2.1-mini and DiffusionGemma, it improves TPF on all four main benchmarks; on LLaDA-2.1-mini it needs only 4.2M self-generated tokens. With DLMRT and NVFP4 quantization, LLaDA-2.1-mini + ORL averages 5,046 tokens/s per single request across HumanEval on one B200 and 2,788 tokens/s on a consumer RTX 5090, and DiffusionGemma + ORL averages 4,214 tokens/s across GSM8K on one B200, 1,540 tokens/s on an RTX 5090, and 7,708 tokens/s on eight B200s.

References

AIME. AIME problems and solutions. https://artofproblemsolving.com/wiki/index.php/AIME_Problems_and_Solutions, 2024.

Diogo Almeida. Introducing system one models & Jev. https://typesafe.ai/blog/introducing-system-one-models-and-jev, September 2026. TypeSafe AI.

Ant Group DeepXPU Team and SGLang Team. Power up diffusion LLMs: Day-0 support for LLaDA 2.0. https://www.lmsys.org/blog/2025-12-19-diffusion-llm/, 2025. LMSYS Org blog.

Marianne Arriola, Aaron Gokaslan, Justin T Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. *arXiv preprint arXiv:2503.09573*, 2025.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Tiwei Bie, Maosong Cao, Xiang Cao, Bingsen Chen, Fuyuan Chen, Kun Chen, Lun Du, Daozhuo Feng, Haibo Feng, Mingliang Gong, et al. LLaDA2.1: Speeding up text diffusion via token editing. *arXiv preprint arXiv:2602.08676*, 2026.

Jian Chen, Yesheng Liang, and Zhijian Liu. DFlash: Block diffusion for flash speculative decoding, 2026a. URL https://arxiv.org/abs/2602.06036.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Shirui Chen, Jiantao Jiao, Lillian J Ratliff, and Banghua Zhu. dUltra: Ultra-fast diffusion language models via reinforcement learning. *arXiv preprint arXiv:2512.21446*, 2025.

Ting Chen, Ruixiang Zhang, and Geoffrey Hinton. Analog bits: Generating discrete data using diffusion models with self-conditioning. *arXiv preprint arXiv:2208.04202*, 2022.

Zigeng Chen, Gongfan Fang, Xinyin Ma, Ruonan Yu, and Xinchao Wang. DMax: Aggressive parallel decoding for dLLMs. *arXiv preprint arXiv:2604.08302*, 2026b.

Zigeng Chen, Gongfan Fang, Xinyin Ma, Ruonan Yu, and Xinchao Wang. dParallel: Learnable parallel decoding for dLLMs. In *International Conference on Learning Representations*, volume 2026, pp. 53193–53211, 2026c.

Shuang Cheng, Yihan Bian, Dawei Liu, Linfeng Zhang, Qian Yao, Zhongbo Tian, Wenhai Wang, Qipeng Guo, Kai Chen, Biqing Qi, et al. SDAR: A synergistic diffusion-autoregression paradigm for scalable sequence generation. *arXiv preprint arXiv:2510.06303*, 2025.

Xin Cheng, Xingkai Yu, Chenze Shao, Jiashi Li, Yunfan Xiong, Yi Qian, Jiaqi Zhu, Shirong Ma, Xiaokang Zhang, Jiasheng Ye, et al. DSpark: Confidence-scheduled speculative decoding with semi-autoregressive generation. *arXiv preprint arXiv:2607.05147*, 2026.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, et al. The entropy mechanism of reinforcement learning for reasoning language models. *arXiv preprint arXiv:2505.22617*, 2025.

Phuong Tuan Dat, Qi Li, and Xinchao Wang. dOPSD: On-policy self-distillation for diffusion language models. *arXiv preprint arXiv:2607.04428*, 2026.

Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. *arXiv preprint arXiv:1807.03819*, 2018.

DiffusionGemma Team, Adrien Ali Taïga, James Assiene, Daniele Calandriello, Rahma Chaabouni, João Gante, Tamara von Glehn, Nate Keating, Chris Knutsen, Martin Kukla, et al. DiffusionGemma technical report. *arXiv preprint arXiv:2608.00146*, 2026.

Charles Frye and Shankha Biswas. Achieve state-of-the-art inference latencies with speculative decoding. https://modal.com/blog/achieve-sota-specdec, June 2026. June 24, 2026.

Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, Tongkai Yang, Binhang Yuan, and Yi Wu. AReaL: A large-scale asynchronous reinforcement learning system for language reasoning. *arXiv preprint arXiv:2505.24298*, 2025.

Yonggan Fu, Lexington Whalen, Abhinav Garg, Chengyue Wu, Maksim Khadkevich, Nicolai Oswald, Enze Xie, Daniel Egert, Sharath Turuvekere Sreenivas, Shizhe Diao, et al. Nemotron-Labs-Diffusion: A tri-mode language model unifying autoregressive, diffusion, and self-speculation decoding. *arXiv preprint arXiv:2607.05722*, 2026.

Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. *Advances in Neural Information Processing Systems*, 38: 41340–41391, 2025a.

Jonas Geiping, Xinyu Yang, and Guinan Su. Efficient parallel samplers for recurrent-depth models and their connection to diffusion language models. *arXiv preprint arXiv:2510.14961*, 2025b.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. *arXiv preprint arXiv:2103.03874*, 2021.

Yanzhe Hu, Yijie Jin, Pengfei Liu, Kai Yu, and Zhijie Deng. LightningRL: Breaking the accuracy-parallelism trade-off of block-wise dLLMs via reinforcement learning. *arXiv preprint arXiv:2603.13319*, 2026a.

Yuezhou Hu, Harman Singh, Monishwaran Maheswaran, Haocheng Xi, Coleman Hooper, Jintao Zhang, Aditya Tomar, Michael W Mahoney, Sewon Min, Mehrdad Farajtabar, et al. Residual context diffusion language models. *arXiv preprint arXiv:2601.22954*, 2026b.

Jianuo Huang, Yaojie Zhang, Qituan Zhang, Hao Lin, Hanlin Xu, and Linfeng Zhang. Domino: Decoupling causal modeling from autoregressive drafting in speculative decoding. *arXiv preprint arXiv:2605.29707*, 2026.

Allan Jabri, David Fleet, and Ting Chen. Scalable adaptive computation for iterative generation. *arXiv preprint arXiv:2212.11972*, 2022.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.

Ahmadreza Jeddi, Marco Ciccone, and Babak Taati. LoopFormer: Elastic-depth looped transformers for latent reasoning via shortcut modulation. *arXiv preprint arXiv:2602.11451*, 2026.

Haozhe Jiang, Haiwen Feng, Pieter Abbeel, Jiantao Jiao, Angjoo Kanazawa, and Nika Haghtalab. Diffusion policy optimization without drifting apart. *arXiv preprint arXiv:2606.13795*, 2026.

Renren Jin, Pengzhi Gao, Yuqi Ren, Zhuowen Han, Tongxuan Zhang, Wuwei Huang, Wei Liu, Jian Luan, and Deyi Xiong. Revisiting entropy in reinforcement learning for large reasoning models. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 25300–25322, 2026.

Mingyu Jo, Jaesik Yoon, Justin Deschenaux, Caglar Gulcehre, and Sungjin Ahn. Loopholing discrete diffusion: Deterministic bypass of the sampling wall. In *International Conference on Learning Representations*, volume 2026, pp. 93890–93918, 2026.

Marius Killinger and Philip Kiely. Why GPU utilization matters for model inference. https://www.baseten.co/blog/why-gpu-utilization-matters-for-model-inference/, 2025. Baseten, last updated May 18, 2025.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th symposium on operating systems principles*, pp. 611–626, 2023.

Sanghyun Lee, Chunsan Hong, Seungryong Kim, Jonghyun Lee, Jongho Park, and Dongmin Park. Looped diffusion language models. *arXiv preprint arXiv:2605.26106*, 2026.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. *arXiv preprint arXiv:2503.01840*, 2025.

Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

Xiaocheng Lu, Hualei Zhang, Shuhan Guo, Jie Zhang, Xiaoyi Pang, Jian Liu, Haoxi Li, Bohai Gu, Haoxuan Che, Jingcai Guo, et al. OPTD: On-policy transition distillation with consistency-guided adaptive compression for few-step diffusion language models. *arXiv preprint arXiv:2608.02942*, 2026.

Yifu Luo, Zeyu Chen, Haoyu Wang, Xinhao Hu, Yuxuan Zhang, Zhizhou Sha, and Shiwei Liu. Learning from the self-future: On-policy self-distillation for dLLMs. *arXiv preprint arXiv:2606.18195*, 2026.

Nebius team. AI model components: Key elements of a GenAI inference setup explained. https://nebius.com/blog/posts/ai-model-components-key-elements-of-a-genai-inference-setup-explained, August 2025. August 5, 2025.

Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. *arXiv preprint arXiv:2502.09992*, 2025.

NVIDIA. DiffusionGemma 26B A4B IT NVFP4. Hugging Face model card, 2026. URL https://huggingface.co/nvidia/diffusiongemma-26B-A4B-it-NVFP4.

NVIDIA Model Optimizer Team. Improving NVFP4 accuracy with local-Hessian weight scales. NVIDIA Model Optimizer, September 2026. URL https://nvidia.github.io/Model-Optimizer/announcements/local-hessian.html.

Yu-Yang Qian, Junda Su, Lanxiang Hu, Peiyuan Zhang, Zhijie Deng, Peng Zhao, and Hao Zhang. d3LLM: Ultra-fast diffusion LLM using pseudo-trajectory distillation. *arXiv preprint arXiv:2601.07568*, 2026.

David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. GPQA: A graduate-level Google-proof Q&A benchmark. In *First conference on language modeling*, 2024.

Yuji Ren, Chenkai Xu, Zhuocheng Gong, Jianguo Li, and Zhijie Deng. CForce: Boosting parallel decoding for dLLMs via consistency forcing. *arXiv preprint arXiv:2608.13925*, 2026.

Shane Robbins. Production AI runs on inference. Are you ready for it? https://www.coreweave.com/blog/production-ai-runs-on-inference-are-you-ready-for-it, June 2026. CoreWeave.

Benjamin Rozonoyer, Jacopo Minniti, Dhruvesh Patel, Neil Band, Avishek Joey Bose, Tim GJ Rudner, and Andrew McCallum. Learned relay representations for forward-thinking discrete diffusion models. *arXiv preprint arXiv:2605.22967*, 2026.

Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J Reddi. Reasoning with latent thoughts: On the power of looped transformers. In *International Conference on Learning Representations*, volume 2025, pp. 14855–14881, 2025.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A flexible and efficient RLHF framework. *arXiv preprint arXiv:2409.19256*, 2024.

Xu Wang, Chenkai Xu, Yijie Jin, Jiachun Jin, Hao Zhang, Kai Yu, and Zhijie Deng. Diffusion LLMs can do faster-than-AR inference via discrete diffusion forcing. In *International Conference on Learning Representations*, volume 2026, pp. 2282–2299, 2026a.

Yinjie Wang, Ling Yang, Bowen Li, Ye Tian, Ke Shen, and Mengdi Wang. Revolutionizing reinforcement learning framework for diffusion large language models. In *International Conference on Learning Representations*, volume 2026, pp. 101476–101501, 2026b.

Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, et al. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. *Advances in Neural Information Processing Systems*, 37:95266–95290, 2024.

Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. Fast-dLLM: Training-free acceleration of diffusion LLM by enabling KV cache and parallel decoding. *arXiv preprint arXiv:2505.22618*, 2025.

Chengyue Wu, Hao Zhang, Shuchen Xue, Shizhe Diao, Yonggan Fu, Zhijian Liu, Pavlo Molchanov, Ping Luo, Song Han, and Enze Xie. Fast-dLLM v2: Efficient block-diffusion LLM. In *International Conference on Learning Representations*, volume 2026, pp. 128353–128370, 2026.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7B: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025.

Yifan Yu, Yuqing Jian, Junxiong Wang, Zhongzhu Zhou, Donglin Zhuang, Xinyu Fang, Sri Yanamandra, Xiaoxia Wu, Qingyang Wu, Shuaiwen Leon Song, Tri Dao, Ben Athiwaratkun, James Zou, Fan Lai, and Chenfeng Xu. Introspective diffusion language models. *arXiv preprint arXiv:2604.11035*, 2026.

Siyan Zhao, Devaansh Gupta, Qinqing Zheng, and Aditya Grover. d1: Scaling reasoning in diffusion large language models via reinforcement learning. *Advances in Neural Information Processing Systems*, 38:56729–56762, 2025.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody H Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. SGLang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37:62557–62583, 2024.

Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023.

Rui-Jie Zhu, Zixuan Wang, Kai Hua, Tianyu Zhang, Ziniu Li, Haoran Que, Boyi Wei, Zixin Wen, Fan Yin, He Xing, et al. Scaling latent reasoning via looped language models. *arXiv preprint arXiv:2510.25741*, 2025.

A Related Works

Residual diffusion language models. Retaining continuous information across denoising steps lets a DLM reuse computation beyond its discrete token commitments, building on self-conditioning in continuous diffusion (Chen et al., 2022; Jabri et al., 2022). Loopholing (Jo et al., 2026) introduces a deterministic latent pathway that bypasses the information loss of categorical sampling; RCD (Hu et al., 2026b) constructs entropy-weighted residual embeddings from predictive distributions and trains the model with reference-generated residual signals; and DiffusionGemma (DiffusionGemma Team et al., 2026) incorporates distribution-based self-conditioning into blockwise generation. Closely related, Relay (Rozonoyer et al., 2026) trains a cross-step hidden-state channel through truncated backpropagation through time, committing ground-truth token values during its training rollouts. These approaches establish the value of cross-step feedback, but their training states are typically constructed from ground-truth sequences or reference-generated signals. We identify a further challenge: as the model changes, both its residual predictions and the trajectories they induce change together. Our multi-seed comparison of every-step and infrequent rollout refresh shows distinct learning curves (Figure 2(a)). To track the evolving distribution, ORL adopts a fully on-policy framework: every update uses newly generated trajectories from the current model, retaining verified responses and replaying intermediate states with the exact residual inputs consumed during generation. Adjacent entropy contraction encourages faster refinement, while our co-designed training system makes this workflow practical at scale.

Reinforcement learning and on-policy post-training for DLMs. d1 (Zhao et al., 2025) adapts policy-gradient optimization to masked DLMs, while DiPOD (Jiang et al., 2026) introduces on-policy ELBO regularization to mitigate drift between its ELBO surrogate and the true log-likelihood. dUltra (Chen et al., 2025) and LightningRL (Hu et al., 2026a) target decoding efficiency through learned unmasking decisions and efficiency-aware rewards. However, generating responses on-policy does not by itself ensure that training preserves the states encountered during decoding: LightningRL's released implementation constructs block-masked training inputs from completed responses rather than replaying the intermediate decoding states. TraceRL (Wang et al., 2026b) aligns training with the decoding trace, preserving step-level conditioning when no trajectory shrinkage is applied, but its policy objective supervises the token actions committed at each step rather than predictions at positions that remain unresolved. Related non-RL approaches also exploit model-generated trajectories: d-OPSD (Luo et al., 2026) constructs a self-teacher by selectively revealing tokens from generated completions, dOPSD (Dat et al., 2026) distills later-state predictions into earlier states, and OPTD (Lu et al., 2026) learns compressed transitions on student-visited states. ORL combines exact residual-state replay with supervision beyond the sampled actions. At selected states of verified rollouts, it supervises intermediate predictions toward the final response, including at positions not committed at that step. Adjacent entropy contraction further encourages uncertainty reduction over the unresolved canvas between consecutive steps, providing a differentiable signal for denoising progress rather than relying solely on trajectory-level efficiency rewards. Both objectives operate within a fully on-policy framework that regenerates trajectories after every update and preserves the exact residual context consumed at each training state.

Looped Transformers and latent computation. Recurrent computation offers a way to increase reasoning depth without proportionally increasing model parameters. Universal Transformers (Dehghani et al., 2018) introduce shared transformations with adaptive computation, and subsequent work studies their reasoning benefits (Saunshi et al., 2025) and scales latent recurrence to language modeling (Geiping et al., 2025a; Zhu et al., 2025). LoopFormer (Jeddi et al., 2026) further uses shortcut-consistency training to support variable computation budgets. This perspective connects naturally to diffusion generation: Geiping et al. (2025b) connect recurrent-depth models to diffusion through parallel sampling, while LoopMDM (Lee et al., 2026) introduces selective layer recurrence within masked diffusion models. Residual DLMs realize recurrent computation through continuous predictive feedback between denoising steps, without requiring additional layer repetitions within each forward pass. ORL is complementary to these architectural advances: it trains the cross-step feedback on fresh trajectories from the current model, with the exact residual context encountered during generation, and combines verified-output supervision with an explicit refinement objective so that intermediate predictions support accurate outputs with fewer decoding steps.

B Residual Diagnostics

B.1 Predictive Information and Capture Cost

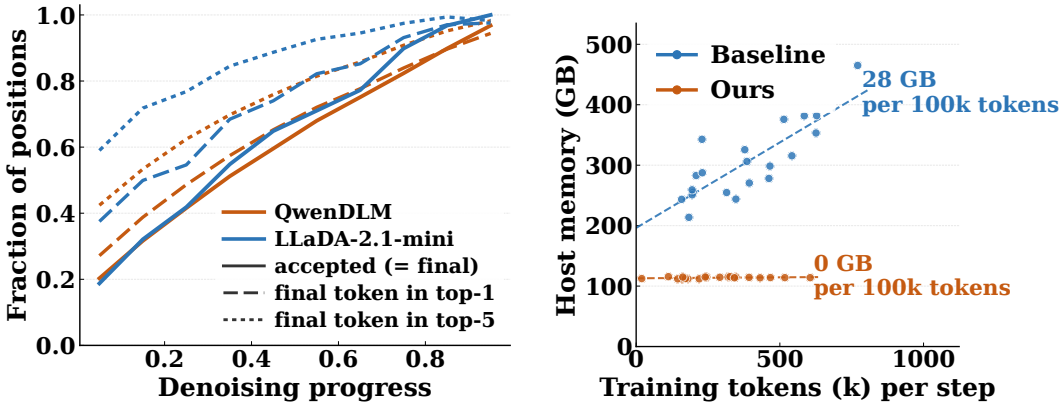
Intermediate predictions often contain the final decoded token before it is committed (Figure 10a). This is agreement with the trajectory's output, not reference-answer correctness. Retaining such states also creates a memory bottleneck: naive capture adds about 28 GB of host memory per 100k training tokens, versus 36 GB for the combined host/GPU footprint in Figure 2(c).

C Optimization Details and Stabilization

Response normalization. For verified responses $\mathcal{R}_{\mathcal{B}}^{+}$, retained states $\mathcal{S}(\mathcal{R})$, and valid supervised positions $\mathcal{I}_b^{\mathcal{R}}$ (excluding padding and positions after terminal EOS), let $N_{\mathcal{R}} = \sum_{(b,k)\in\mathcal{S}(\mathcal{R})} |\mathcal{I}_b^{\mathcal{R}}|$. QwenDLM and LLaDA use

$$\mathcal{A}[\ell] = \frac{1}{|\mathcal{R}_{\mathcal{B}}^{+}|} \sum_{\mathcal{R}\in\mathcal{R}_{\mathcal{B}}^{+}} \frac{1}{N_{\mathcal{R}}} \sum_{(b,k)\in\mathcal{S}(\mathcal{R})} \sum_{i\in\mathcal{I}_b^{\mathcal{R}}} \ell_{\mathcal{R},b,k,i}. \tag{3}$$

Thus each verified response has equal token-loss weight; empty verified batches are skipped.



(a) Intermediate top-k agreement with final tokens.

(b) Host memory versus materialized training tokens.

Figure 10: Residual-learning motivation. Left: top-1/top-5 agreement exceeds the accepted-and-final fraction early in denoising. Right: sparse, GPU-resident capture avoids the baseline's growing host footprint over the measured range.

AEC aggregation. A valid pair $(b, k) \in \mathcal{P}(\mathcal{R})$ has a selected target state $(b, k+1)$ and nonempty unresolved sets at both states. The source need not be selected: its captured mean normalized entropy supplies the reference. Give each pair weight $w_{\mathcal{R}} = 1/|\mathcal{P}(\mathcal{R})|$ before gating. QwenDLM uses $g_{\mathcal{R},b,k} = \mathbf{1}\{H^{\text{roll}}_{b,k} \le 0.20\}$; LLaDA uses $g = 1$. With $D = \sum_{\mathcal{R},(b,k)\in\mathcal{P}(\mathcal{R})} w_{\mathcal{R}} g_{\mathcal{R},b,k}$,

$$\mathcal{L}_{\text{AEC}} = \frac{\sum_{\mathcal{R},(b,k)\in\mathcal{P}(\mathcal{R})} w_{\mathcal{R}} g_{\mathcal{R},b,k} \ell^{\text{AEC}}_{\mathcal{R},b,k}}{D}, \qquad D > 0, \tag{4}$$

and the loss is zero for $D = 0$. Responses with no valid pair are excluded. Weights are not recomputed after gating: a response's remaining mass is its retained fraction of pairs. DiffusionGemma instead uses the canvas-pair mean in Appendix E.4.

Entropy-gated supervision. For detached reference q_i^{cap}, replayed distribution $\widehat{p}_i$, normalized entropy $\overline{H}$, and cap C,

$$\ell_i^{\text{hybrid}} = \begin{cases} -\log \widehat{p}_i(y_i), & \overline{H}(q_i^{\text{cap}}) > \zeta, \\ \min\{D_{\text{KL}}(\widehat{p}_i \| q_i^{\text{cap}}), C\}, & \text{otherwise.} \end{cases} \tag{5}$$

Only QwenDLM's final training stage uses this gate, with $\zeta = 0.10$ and $C = 5$; all other reported training stages use CE. The teacher is reconstructed from each state's captured, shifted output-head-input hidden vector using the *current* output head under stop-gradient, not an old head or one final-response teacher. Both head outputs are cast to float32, divided by loss temperature 1, clipped to $[-50, 50]$, then normalized over the vocabulary. Replay gradients accumulate before one optimizer step.

For $p = \text{softmax}(a)$ and $D_{\text{KL}}(p\|q) = D$, $\partial D/\partial a_j = p_j[\log(p_j/q_j) - D]$ before capping. Identical transformed distributions therefore give zero RKL loss and gradient. Exact captured inputs do not guarantee rollout/replay numerical parity; any nonzero RKL contribution in this training recipe was not measured separately from the effect of removing low-entropy CE.

Residual initialization. QwenDLM's bias-free adapter is $g_\phi(z) = W_d[\text{SiLU}(W_g u) \odot W_u u]$, $u = \text{RMSNorm}_w(z)$, with hidden/intermediate dimensions 2,560/9,728 and epsilon 10^{-6}. Multiplying all three matrices by $\gamma = 8$ and dividing both the normalized input and branch output by γ preserves its function:

$$\widetilde{g}(z) = \frac{\gamma W_d}{\gamma}[\text{SiLU}(\gamma W_g u/\gamma) \odot (\gamma W_u u/\gamma)] = g_\phi(z). \tag{6}$$

The RMSNorm gain is unchanged. Both compensation constants are $1/8$, distinct from injection strength $\alpha = 1$; the residual sum then receives non-affine RMS normalization scaled by 0.02. The same post-normalization is applied during replay to z in Equation 1. Compensation is applied once,

before initializing the ORL optimizer, not again on resume. Output-only compensation would not preserve SiLU's arguments.

LLaDA uses affine pre-RMSNorm and three bias-free gated-MLP projections; activation, dimensions, and normalization epsilon follow the model config. It initializes gate/up weights from a Gaussian with standard deviation 10^{-3}, down weights to zero, and pre-RMSNorm gain to one. All adapter parameters use 0.3 times the backbone learning rate. For LLaDA, self-conditioning probability and injection strength are both fixed at one; no injection ramp or post-normalization is used. Initialization is checked after meta-materialization and before FSDP synchronization. DiffusionGemma retains its native self-conditioning module.

D System Implementation Details

Figure 3 summarizes the ORL loop and where each mechanism below acts.

D.1 Asynchronous Exact-State Capture

Each forward snapshots mutable token IDs and accept/edit masks, retains the residual and required GPU statistics, records a CUDA event, and enqueues a bounded handle. A background worker materializes the record after the event; the decode critical path performs no host copy or host synchronization. Exhausting the queue's entry/byte budget blocks for a bounded interval or fails explicitly, never silently dropping states. Host-bound fields use pinned memory. With a colocated trainer, residual/hidden-source pages can remain GPU-resident and cross through one-shot CUDA-IPC handles, relaying only required pages to another GPU. Storage remains live until consumer acknowledgement.

D.2 Staircase Attention

A noisy block attends to the prompt, preceding clean blocks, and itself, but not its clean target, later blocks, other noisy blocks, or other documents. Clean queries never attend to noisy keys. The Triton kernel derives visible key ranges from per-document metadata, skips invisible tiles, and applies the predicate within boundary tiles; it allocates no dense $N \times N$ mask. Backward uses the same visibility: dQ follows the query traversal, while dK, dV derive valid query ranges per key tile. The implementation supports variable-length packed documents, grouped-query attention, and sequence parallelism. Our code repository contains the version of this kernel used to train and reproduce our models.[2]

D.3 Packing and Cost Balancing

Same-round states share clean context, following the slicing structure of TraceRL (Wang et al., 2026b); residual/source pages remain sparse and block-local. Replay rows are packed as mutually invisible documents. A weighted proxy of sequence tokens, supervised positions, and resident pages drives deterministic largest-cost-first assignment to data-parallel groups. Chunked materialization bounds the live input/page footprint. Under hybrid underfill, splittable label sets are partitioned before zero-loss ghost rows align collective schedules. Each label has one owner, and global reduction uses $\mathcal{L} = \sum_j \mathrm{num}_j / \sum_j \mathrm{den}_j$, preserving the unpacked objective up to rounding.

E Training and Evaluation Details

E.1 Evaluation Protocols

Measured throughput uses one H100, TP=1, and a single request in flight. Length-terminated responses count as incorrect and are excluded only from clean efficiency statistics. Request time includes prefill, decoding, scheduling, and host overhead, but excludes model loading, server startup,

[2]The latest version is maintained at https://github.com/yuezhouhu/fast-diffusion-attention, which provides kernels derived from FlashAttention-2 and FlashAttention-3 and matches FlashAttention's numerical accuracy on SM80, SM86, SM89, and SM90 GPUs.

and preprocessing. Warmup exclusions are stated per run. Within each family, Base and ORL share tasks and scorers. Each main-table evaluation record is associated with its checkpoint and evaluation profile in the code repository (Appendix E.1). Runtime acceleration is evaluated separately in Appendix F.

Runtime settings. QwenDLM and LLaDA use SGLang (Zheng et al., 2024); DiffusionGemma uses vLLM (Kwon et al., 2023) in BF16. Reproducing the throughput results of Figure 1(b), Section 4.3, and Figure 13 requires DLMRT (Section 3.3). Engine revisions, container digests, attention backends, cache controls, and graph settings are recorded in `evaluation/configs/paper/decoding/` in the code repository. Absolute TPS is hardware/runtime-specific. Separate B200 NVFP4 measurements, including the quantization procedure and FP8 KV-cache configuration, are reported in Appendix F.2.

Tasks and scoring. The main suite uses GSM8K test (1,319 tasks) (Cobbe et al., 2021), MATH-500 (500) (Hendrycks et al., 2021), HumanEval (164, base tests) (Chen et al., 2021), and the canonical MBPP427 collection (Austin et al., 2021) with its original multi-candidate extraction. Base/ORL share task and scorer protocols within each family; cross-family and literature comparisons are not scorer-matched. QwenDLM retains its CodeTACO code-scoring path, while LLaDA/DiffusionGemma use EvalPlus base tests for HumanEval. In the code repository, `evaluation/repro/paper/main.yaml` and `evaluation/repro/eval/portable/protocols.json` record input identities, native prompting, extraction rules, and scorer pins.

Runs and aggregation. For each ORL configuration, we conduct multiple independent training runs and repeated evaluations of each resulting checkpoint. In the main benchmark suite, the same checkpoint from each training run is evaluated across all benchmarks. We first average the evaluation metrics within each checkpoint, then average equally across independent training runs. This averaging protocol also applies to the component and hyperparameter ablations in Table 3. Starting-model baselines average repeated evaluations of their reference checkpoints. Each evaluation generates one completion per task; DiffusionGemma uses three generation-seed settings per checkpoint. Seed settings and task partitions are recorded in `evaluation/repro/paper/main.yaml`; transfer-evaluation settings are given below. Reported means alone do not establish statistical significance for small score differences. Within an individual HumanEval evaluation, one task corresponds to approximately 0.61 percentage points; averaging repeated evaluations permits smaller increments in the reported means.

Reproduction configurations. For the complete generation configuration associated with each main-table checkpoint and operating point, refer to our code repository. The experiment index at `evaluation/configs/paper/experiments.yaml` maps results to their profiles in `evaluation/configs/paper/decoding/`; `evaluation/docs/reproduction/SETTINGS.md` provides the configuration guide. Profiles outside the main table are defined where they are reported (Tables 13 and 15; Appendix G.4).

Accounting. Let $\mathcal{I}_b$ be all requests, $\mathcal{C}_b$ those not length-terminated, s_i the scorer outcome, C_i completion tokens, D_i denoising forwards, and W_i complete request time. We report

$$\mathrm{Acc}_b = \frac{100}{|\mathcal{I}_b|} \sum_{i \in \mathcal{C}_b} s_i, \qquad \mathrm{TPF}_b = \frac{1}{|\mathcal{C}_b|} \sum_{i \in \mathcal{C}_b} \frac{C_i}{D_i}, \tag{7}$$

$$\mathrm{TPS}_b^{\mathrm{clean}} = \frac{\sum_{\mathcal{C}_b} C_i}{\sum_{\mathcal{C}_b} W_i}, \qquad \mathrm{TPS}_b^{\mathrm{all}} = \frac{\sum_{\mathcal{I}_b} C_i}{\sum_{\mathcal{I}_b} W_i}, \tag{8}$$

$$\mathrm{TPS}_b^{\mathrm{completed}} = \frac{\sum_{\mathcal{C}_b} C_i}{\sum_{\mathcal{I}_b} W_i}. \tag{9}$$

Completed-output TPS charges failed-request time without crediting its tokens, but still credits completed incorrect answers; it is not correct-answer throughput. All-forward TPF replaces D_i by $F_i = D_i + R_i + K_i$ (denoise, refinement, commit). Wall TPS includes all this work; the approximation $\mathrm{TPF}\,/t_{\mathrm{forward}}$ does not. We compute benchmark metrics for each evaluation and average

Table 8: Transfer evaluation. Evaluation-seed counts are generation-seed settings per trained checkpoint (LL: LLaDA; DG: DiffusionGemma). ORL metrics average independent training runs and repeated evaluations; starting-model baselines average repeated evaluations of their reference checkpoints. Each evaluation generates one completion per task. AIME-24 scores 30 tasks from a shared 60-task, two-year generation input.

Benchmark	Tasks	Eval. seeds LL/DG
AIME-24	30	8 / 8
GPQA-Diamond	198	3 / 3
MMLU-Pro	12,032	1 / 3
LiveCodeBench-v6	175	1 / 3
IFEval	541	1 / 3

repeated evaluations within each checkpoint. For ORL, we then average equally across independent training runs. Cross-benchmark summaries take an equal-weight mean of the benchmark-level metrics.

Baseline sources. The underlined starting points and ORL rows in Table 1 are ours; other rows are paper-reported. Fast-dLLM–LLaDA (Wu et al., 2025), dParallel–LLaDA (Chen et al., 2026c), D2F–LLaDA (Wang et al., 2026a), Fast-dLLM-v2-7B (Wu et al., 2026), and d3LLM follow the unified d3LLM evaluation (Qian et al., 2026), whose MATH entries are MATH (4-shot) on the full test set rather than MATH-500; SDAR-8B-b32 (Cheng et al., 2025) follows LightningRL (Hu et al., 2026a). RCD (Hu et al., 2026b), dUltra-coding (Chen et al., 2025), and DMax-Math (Chen et al., 2026b) use their original papers. TraDo-4B (dynamic) is the TraceRL-trained model reported by Wang et al. (2026b). For dOPSD (Dat et al., 2026), we quote the LLaDA-8B-Instruct results from Table 1: GSM8K/MATH-500 accuracy $72.87/36.00$ and HumanEval/MBPP pass@1 $39.63/39.54$, with math/code averages $54.44/39.59$. Corresponding TPF is unreported and marked with dashes. These published comparisons retain their original backbones and evaluation settings; they are not scorer-matched reruns.

Derived comparison points are estimates, not measured wall TPS. Their conversion is $\widehat{\mathrm{TPS}}_{m,b} = c_b \mathrm{TPF}_{m,b}$, with c_b taken from a measured reference's TPS/TPF or the mean of two matched endpoint ratios. Disaggregating a paper's aggregate TPS instead uses $\widehat{\mathrm{TPS}}_{m,b} = \mathrm{TPS}_m^{\mathrm{paper}} \mathrm{TPF}_{m,b} / \overline{\mathrm{TPF}}_m$ under that paper's task, hardware, and decoding protocol.

E.2 Additional Benchmarks

Table 8 summarizes the transfer suite: AIME-24 (AIME, 2024), GPQA-Diamond (Rein et al., 2024), MMLU-Pro (Wang et al., 2024), LiveCodeBench-v6 (Jain et al., 2024), and IFEval (Zhou et al., 2023). In the code repository, `evaluation/repro/paper/transfer.yaml` and `evaluation/repro/paper/TRANSFER_GENERATION.md` record exact seeds, input hashes, prompt serialization, context settings, and scorer profiles.

Quality and clean per-response TPF are first averaged over repeated evaluations of each checkpoint; ORL metrics are then averaged equally across independent training runs. Quality is not best-of-k or majority voting. AIME uses numeric parsing, GPQA/MMLU-Pro the original multiple-choice parser, LiveCodeBench execution tests, and IFEval its instruction evaluator. Historical source/RNG binding for the LCB/IFEval reproduction scorers and the DiffusionGemma long-task context override remains incomplete; the repository records these separately from fresh-run settings.

Transfer quality. In Table 2, TPF increases on all ten model–benchmark pairs, while accuracy changes vary by task. LLaDA's accuracy declines by 3.4, 1.8, and 1.7 percentage points on GPQA-Diamond, IFEval, and AIME-24, respectively; DiffusionGemma's accuracy declines by 2.1 percentage points on LiveCodeBench-v6.

E.3 Offline SDAR Initialization

QwenDLM starts from a non-residual Qwen3-4B-Thinking-2507 SDAR checkpoint. We add a zero-output residual adapter and train with CE on ReasonMix/SFTMix, using block size 256 and an 8,192-token cutoff. The initialization checkpoint for the on-policy experiments and the full offline recipe, including optimizer, normalization, and hardware settings, are recorded in `evaluation/configs/paper/training.yaml`; the runnable offline recipes are in `offline/recipes/`. An auxiliary sweep evaluated 118 checkpoints on GSM8K100 and HumanEval164; later checkpoints did not monotonically improve accuracy.

E.4 On-Policy Training Recipes

We train each model until tokens per forward (TPF) stabilizes. Each iteration draws one prompt and eight rollouts and updates on verified responses only; batches with no verified response do not trigger an optimizer update. `evaluation/configs/paper/training.yaml` in the code repository records the complete per-model recipes, training schedules, checkpoint identities, rollout controls, adapter initialization, optimizer settings, resume records, and compute accounting; the runnable recipes are in `training/recipes/`.

Data. QwenDLM uses 819 OpenCodeInstruct-derived Python problems. LLaDA and DiffusionGemma use a balanced mixture of those code problems, 819 GSM8K problems, and 819 MATH problems. Code rewards use the supplied test suite; math rewards use task-specific verifiers. Exact normalized checks found no final-training-file matches against the corresponding HumanEval, GSM8K, and MATH-500 evaluation sets; semantic and earlier-stage contamination are not established by this check.

QwenDLM. After residual rescaling by $\gamma = 8$, we train at learning rate 10^{-5}, then continue at 5×10^{-6} with the entropy-gated CE/RKL loss. Response-normalized CE is combined with AEC (coefficient 50, $\rho = 0.05$, source-entropy ceiling 0.20), using the state selection in Section 3.

LLaDA-2.1-mini. We add the zero-output adapter and train with block size 32, backbone/adapter learning rates $10^{-5}/3 \times 10^{-6}$, and response-normalized CE plus AEC (coefficient 50, $\rho = 0.20$), without entropy-gated RKL. Length-truncated responses are rejected.

DiffusionGemma. We retain native self-conditioning and train with a 256-token canvas, keeping every intermediate state. Gradients pass through the current canvas forward; captured residuals and prefix-cache construction are detached. Token-mean CE is combined with the mean over valid adjacent canvas pairs within the same block:

$$\mathcal{L}_{\mathrm{AEC}}^{\mathrm{DG}} = \frac{1}{|\mathcal{P}_{\mathrm{DG}}|} \sum_{(t,t+1)\in\mathcal{P}_{\mathrm{DG}}} [H_{t+1}^{\theta} - 0.8\ \mathrm{sg}(H_t^{\theta})]_+^2. \tag{10}$$

Both entropies are recomputed by the actor, and the source is detached. The AEC coefficient is 100, with no entropy gate. AdamW uses a learning rate 10^{-5}. Training uses an eight-GPU actor and a separate eight-GPU rollout worker.

F Inference Optimization

We reduce inference overhead through cache reuse, fused execution, and CUDA graphs (Section 3.3 gives an overview). Implementation details and configuration-specific compatibility checks are provided in the code repository (Appendix E.1). The final output-parity gate covers HE164, EvalPlus MBPP378, and GSM8K100.

Cost and scope. Before the later graph fixes, ORL improves TPF by $1.51\times$ but TPS by $1.23\times$: Base/ORL forwards take 7.47/9.17 ms. After optimization, they take 6.74/8.11 ms. Self-conditioning adds work and fewer forwards amortize fixed costs less well, so TPF is not a throughput substitute. Supported execution modes and compatibility requirements are specified in the repository. Output parity is rechecked on new hardware.

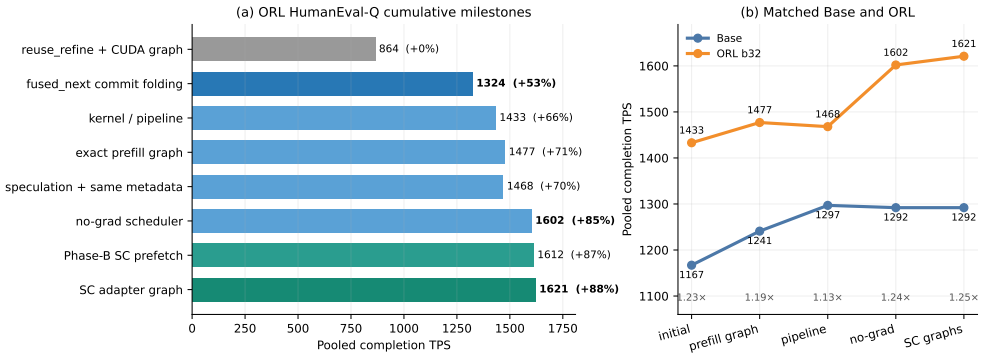


Figure 11: Cumulative LLaDA inference optimizations in SGLang, HumanEval Quality mode. Each side uses four one-H100 engines, pooled completion TPS, and removes the first warmup request per shard. The ORL checkpoint differs from the main-table checkpoint. The 864-to-1,324 transition had 153/164 identical outputs and accuracy $0.896 \rightarrow 0.890$, not exact parity. Later changes pass the final gate. The last two normalized gains are 0.60%/1.18%; their raw paired comparison is $1610.2 \rightarrow 1629.2$ TPS.

F.1 The DLMRT Engine

DLMRT serves one request at a time. A block of 32 tokens (LLaDA-2.1-mini) or 256 tokens (QwenDLM and DiffusionGemma) starts as noise and is refined by denoising rounds until the decoder accepts every position; a causal commit pass then writes the block's keys and values for the blocks after it.

The block graph. DLMRT captures each block as one CUDA graph (Figure 12). The denoising loop is a CUDA conditional `WHILE` node whose condition kernel reads device-side counters after every round, so no round waits for the host: the host launches one graph per block and reads back the block's token IDs. For LLaDA-2.1-mini, a finished block's refinement rounds run as the first half of the next block's first forward pass, which is the commit folding of Section 3.3; DiffusionGemma commits each canvas at the start of the next canvas's graph. DLMRT also runs tensor-parallel across an NVLink node with a barrier-free fused reduction; except for the TP8 row of Table 6, all measurements in this paper use one GPU.

Host, once per block: launch the block's CUDA graph; read back its token IDs; check EOS and the token budget.

Device, one CUDA graph per block:

1: $\mathbf{x} \leftarrow$ prompt tail or noise
2: **while** some position is not accepted **and** round $<$ limit **do** ▷ conditional node
3: self-conditioning from the previous round's probabilities; embedding
4: $L\times$[norm, QKV, Q/K norm and RoPE, KV write, attention, output projection, norm, MLP or MoE]
5: final norm, LM head, row statistics; accept, edit, or re-noise
6: **end while**
7: commit: causal forward pass over the accepted block, writing its keys and values

Figure 12: One block in DLMRT. The denoising loop runs on the GPU inside one CUDA graph; the host only launches the block and reads back its tokens.

Kernels. Every kernel in a round is a programmatically dependent launch: it waits for its predecessor only before touching memory, so its launch overlaps the predecessor's tail (about $0.4\,\mu$s saved per kernel boundary). At 256 rows, the projections sit near the H100's compute–bandwidth ridge (Table 9). DLMRT runs them on DeepGEMM on Hopper, patched with an L2 policy that streams weights and keeps activations resident (1–1.5% faster per round), and on cuBLAS on Blackwell. On Hopper it fuses gate_up and SwiGLU into one ThunderKittens kernel (3–4% per round) and, for QwenDLM, the output projection, residual RMSNorm, gate_up, SwiGLU, and down projection into one persistent kernel (1.6–2.3% per round); all three changes are bit-identical to the unpatched,

unfused kernels. Attention uses FA3 over one contiguous KV arena with device-side sequence lengths, and a Triton flash attention on Blackwell. Q/K normalization, RoPE, and the KV write run as one fused row kernel, and the self-conditioning softmax reuses the decoder's log-normalizer, which cuts it from 412 to 50 μs. MoE routing runs in Triton; the experts use DeepGEMM's masked grouped GEMM in BF16, small-tile Triton grouped GEMMs for LLaDA-2.1-mini's 32-row rounds, and FlashInfer's TRT-LLM-gen NVFP4 MoE on Blackwell (Appendix F.2).

Table 9: Time per call (μs) of the QwenDLM-4B projections at 256 rows on one H100, against the time to read the BF16 weights at peak HBM bandwidth.

Projection	cuBLAS BF16	DeepGEMM BF16	HBM floor
QKV (2560 $\rightarrow$ 6144)	15.7	15.6	9.4
O (4096 $\rightarrow$ 2560)	13.6	12.1	6.3
gate_up (2560 $\rightarrow$ 19456)	46.2	45.1	29.7
down (9728 $\rightarrow$ 2560)	30.8	26.8	14.9

Numerics. DLMRT keeps each model's decoding rule. For QwenDLM and LLaDA-2.1-mini, an SGLang-numerics mode runs SGLang's forward pass and decisions inside DLMRT's block graphs, with kernels that reproduce SGLang's reductions bit for bit where they are faster; Table 5 uses this mode. Against SGLang's records, it reproduces 4,816 of the 4,820 answers of the two QwenDLM models on the four main benchmarks, and all 164 HumanEval answers of LLaDA-2.1-mini with ORL in Speed mode. For DiffusionGemma, every kernel follows the reference implementation's BF16 roundings, and a parity test checks the forward pass against it. Across all three families, accuracy under DLMRT stays within 0.6 points of Table 1.

Latency. Table 10 lists the latency of one denoising round, from self-conditioning to the decoder's decision, at an empty context.

Table 10: Latency of one denoising round in DLMRT at an empty context on one GPU (median over replays of one captured round).

Model	GPU, precision	Round (ms)
QwenDLM + ORL	H100, BF16	5.04
LLaDA-2.1-mini + ORL	H100, BF16	4.99
	B200, NVFP4 experts	2.03
DiffusionGemma + ORL	H100, BF16	16.96
	B200, NVFP4 experts	5.59

F.2 NVFP4 Quantization

To improve inference throughput and latency on Blackwell GPUs, we quantize the ORL models to NVFP4 with 4-bit weights and activations (W4A4). The main challenge is preserving generation quality at this precision. We use NVIDIA's Local-Hessian scale search algorithm (NVIDIA Model Optimizer Team, 2026) to select weight scales that minimize layer-output reconstruction error. For mixture-of-experts (MoE) models, we quantize only the routed experts; attention, routers, shared experts, and all other components remain in their original BF16 precision. Our NVFP4 inference configurations also use an FP8 KV cache. The NVFP4 measurements below use B200 GPUs; the H100 rows repeat the single-H100 BF16 results of the rest of the paper. Section 4.3 summarizes the recipe and results.

Local-Hessian scale search. Consider a linear layer $Y = WX$, where W is the weight matrix and X is the input activations. For reconstructed quantized weights $\hat{W}$, the squared norm of the layer-output reconstruction error is

$$\mathcal{E} = \|(\hat{W} - W)X\|_F^2 = \mathrm{tr}\Big((\hat{W} - W)H(\hat{W} - W)^\top\Big), \qquad H = XX^\top. \tag{11}$$

The input second-moment matrix H is proportional to the Hessian of this reconstruction objective. Unlike weight mean-squared error, this objective accounts for both the importance of individual input features and their correlations.

Model	Configuration	GSM8K	MATH-500	HumanEval	MBPP
LLaDA-2.1-mini-ORL	BF16 (H100)	92.0 / 8.5 / 1281	87.0 / 10.2 / 1627	89.0 / 13.1 / 2124	77.3 / 10.3 / 1539
	NVFP4 (Local-Hessian, B200)	92.0 / 7.54 / 2953	84.0 / 9.72 / 3882	89.0 / 12.23 / 5046	75.9 / 9.79 / 3564
DiffusionGemma-ORL	BF16 (H100)	96.0 / 30.0 / 1456	92.3 / 27.4 / 1415	91.9 / 27.9 / 1378	87.4 / 28.7 / 1361
	NVFP4 (Diagonal-Hessian, B200)	95.6 / 29.25 / 4214	93.4 / 26.69 / 3805	91.9 / 27.32 / 3949	86.3 / 27.96 / 3898
DiffusionGemma	NVFP4 (NVIDIA, B200)	96.1 / 24.81 / 3627	92.7 / 22.79 / 3198	91.3 / 23.91 / 3464	87.1 / 21.60 / 2991

Table 11: BF16 and NVFP4 evaluation of ORL models. Each cell reports accuracy (%) / tokens per forward (TPF) / tokens per second (TPS). Each configuration runs on one GPU of the indicated type, with throughput measured with DLMRT. H100 rows list the single-H100 BF16 results: accuracy and TPF as in Table 1, TPS as in Table 5. B200 rows report NVFP4 measurements; DiffusionGemma results are averaged over generation seeds 1234–1236, with the ORL decoder on all four tasks. NVFP4 configurations use an FP8 KV cache. The throughput gains in Section 4.3 compare NVFP4 on B200 with BF16 on H100.

Local-Hessian makes the search tractable by approximating H as block diagonal and optimizing each 16-weight NVFP4 block independently. For a weight block w_b, it selects

$$s_b^* = \arg\min_{s \in \mathcal{S}} \big(\hat{w}_b(s) - w_b\big)^\top H_b \big(\hat{w}_b(s) - w_b\big), \qquad H_b = X_b X_b^\top \in \mathbb{R}^{16\times 16}, \tag{12}$$

where X_b contains the corresponding input features and $\hat{w}_b(s) = s\, Q_{\mathrm{FP4}}(w_b/s)$ denotes the reconstructed weights. The search evaluates all 126 positive, nonzero FP8 E4M3 scale candidates and chooses the scale with the smallest local output error. This approximation retains correlations within each block while ignoring interactions across blocks. Because the procedure changes only the quantization scales, the resulting weights remain compatible with standard NVFP4 inference kernels and introduce no additional runtime overhead.

Diagonal-Hessian variant. We also evaluate a simpler variant that replaces H_b in Equation 12 with its diagonal matrix. This variant weights reconstruction errors by the second moment of each input feature but ignores all off-diagonal correlations, including those within a 16-weight block. We test both variants and select the quantized checkpoint with the best generation quality. The selected variants are Local-Hessian for LLaDA-2.1-mini-ORL and Diagonal-Hessian for DiffusionGemma-ORL.

Table 11 shows that NVFP4 on B200 increases throughput by 2.31–2.39× for LLaDA-2.1-mini-ORL and 2.69–2.89× for DiffusionGemma-ORL relative to the H100 BF16 results. Accuracy remains broadly comparable, although LLaDA loses 3.0 percentage points on MATH-500 and 1.4 points on MBPP against the H100 results. TPF decreases modestly across all four tasks for both models.

G Throughput and Additional Comparisons

G.1 Full-Task Request Accounting

Table 12 covers the same GSM1319/HE164 IDs on both sides, with no warmup exclusion. Qwen problem text, gold, and prompt-token counts match; wire prompts were not archived. LLaDA lacks prompt text and HE gold tests, so task-ID agreement alone does not establish prompt/test parity.

QwenDLM's benchmark-macro clean TPS is 186.88 for Base and 933.30 for ORL, giving approximately 5× with SGLang; with DLMRT, ORL reaches 1,298 tok/s, about 6.9× the SGLang Base, as reported in the main text, and 4.5× the DLMRT Base (289 tok/s). All-output/completed-output TPS ratios are $4.59/4.70$. At LLaDA's main-table operating points, the clean/all-output/completed-output ratios are $1.23/1.25/1.29$. These values follow the request-accounting definitions in Appendix E.1. The approximately 3× LLaDA result corresponds to the pair with the same observed mean accuracy in Figure 14 (2.95× clean TPS). Qwen Base has one numerically correct but length-terminated GSM answer; counting it as a failure gives $1191/1319 = 90.30\%$ and mean GSM/HE accuracy 86.61% versus 91.03%.

Table 12: Full-task main runs. L/N counts length failures over all tasks. Lengths are mean returned tokens on clean/all requests. TPS columns use Equations 8 and 9: clean, all-output, and completed-output/all-request-time, respectively. Quality always includes all tasks, with length failures incorrect. All runs use SGLang.

Role	Task	Acc. (%)	L/N	Length clean/all	Clean TPS	All TPS	Completed TPS
QwenDLM							
Base	GSM8K	90.30	1/1319	1723/1759	209.7	213.5	209.0
ORL	GSM8K	92.42	1/1319	1140/1164	718.4	724.8	709.3
Base	HumanEval	82.93	6/164	4929/5948	164.1	196.0	156.5
ORL	HumanEval	89.63	4/164	5621/6283	1148.2	1153.7	1007.0
LLaDA-2.1-mini							
Base	GSM8K	91.89	11/1319	265/297	715.7	669.1	592.1
ORL	GSM8K	92.04	2/1319	216/222	949.2	938.7	912.5
Base	HumanEval	86.59	0/164	360/360	1302.3	1302.3	1302.3
ORL	HumanEval	89.02	0/164	546/546	1530.1	1530.1	1530.1

G.2 Speculative-Decoding Comparison

Table 13 reconstructs the GSM8K1319/HumanEval164 comparison in Figure 1b from per-benchmark summaries. Accuracy and TPS each average the two benchmark-level metrics. Within each benchmark, TPS pools tokens and request time over each setting's own clean requests (Equation 8), not a common-clean intersection across settings.

For the Qwen comparison, matched quality uses the main text's one-sided criterion: the fastest measured setting per drafter with mean accuracy at least ORL's minus 0.5 percentage points. At ORL's 91.03%, the corresponding profiles are EAGLE3 QE02 (94.23%, 404.7 TPS) and DFlash QD03 (90.76%, 745.5 TPS); accuracies need not be equal. Speculative TPF is $1 + N_{\text{accepted}}/N_{\text{draft}}$, not denoising TPF.

Configuration. Each server uses one H100 and vLLM, serving a single request at a time; two warmup requests precede measurement. Qwen3-4B-Thinking-2507 uses the drafters `taobao-mnn/Qwen3-4B-Thinking-2507-Eagle` (EAGLE3) and `bingyang-lei/Qwen3-4B-Thinking-Draft-OPD` (DFlash); Gemma-4-26B-A4B-it uses `RedHatAI/gemma-4-26B-A4B-it-speculator.eagle3` (EAGLE3) and `z-lab/gemma-4-26B-A4B-it-DFlash` (DFlash). The Qwen DFlash drafter targets a different Qwen release. These settings are recorded in `evaluation/configs/paper/speculative.yaml` in the code repository.

G.3 Additional Model Comparisons

Figure 13 extends the GSM8K/HumanEval comparison in Figure 1b with additional model families and methods, including TraDo-8B (Wang et al., 2026b). CForce (Ren et al., 2026) and Nemotron-Diff (Fu et al., 2026) use published results converted to TPS; LightningRL (Hu et al., 2026a) uses our measured TPF converted to TPS (Appendix E.1). These converted points indicate approximate throughput positions. The remaining points use our measured wall-clock completion throughput for a single request on one H100.

G.4 Quality–Throughput Sweeps

Figure 14 contains all 48 operating points: ten each for Qwen Base and ORL, and fourteen per LLaDA checkpoint. Qwen uses common GSM IDs 0–399 and HE164; LLaDA uses GSM1319 and HE164. Qwen Base sweeps the acceptance threshold over 0.30, 0.40, 0.50, 0.60, 0.70, 0.80, 0.85, 0.90, 0.95, and 0.99, and Qwen ORL sweeps the entropy budget over 0.2, 0.3, 0.4, 0.5, 0.6, 0.8, 1.0, 1.5, 2.0, and 3.0; each LLaDA checkpoint uses profiles L01–L14 (Table 15).

Table 13: Speculative-decoding results on GSM8K1319 and HumanEval164 (Figure 1b). Accuracy and clean TPS each average the two benchmark-level metrics. Length failures count as incorrect and are excluded from clean TPS; counts cover both tasks. τ is the verification threshold: a drafted token v is accepted when the target model's probability satisfies $p(v) \geq \tau \max_u p(u)$, so $\tau = 1$ is strict verification. High-failure profiles remain visible, but clean TPS does not charge their failed-request time.

Target / method	Profile	τ	Mean acc. (%)	Clean TPS	Length failures
Qwen / AR	QA	—	93.40	240.1	2
Qwen / EAGLE3	QE01	1	93.47	352.7	3
	QE02	0.1	94.23	404.7	3
	QE03	0.03	88.39	432.3	23
	QE04	0.01	81.08	462.6	69
	QE05	0.003	65.58	483.4	220
	QE06	0.001	48.63	467.6	456
Qwen / DFlash	QD01	1	93.44	573.3	3
	QD02	0.1	92.45	690.9	3
	QD03	0.03	90.76	745.5	12
	QD04	0.01	87.15	788.6	32
	QD05	0.003	80.37	827.1	75
Gemma / AR	GA	—	96.76	206.8	2
Gemma / DFlash	GD01	1	96.80	549.1	1
	GD02	0.03	97.47	605.9	2
	GD03	0.003	96.83	634.0	3
	GD04	10^{-4}	95.77	639.8	10
	GD05	10^{-5}	94.20	662.7	21
Gemma / EAGLE3	GE01	1	96.83	331.1	0
	GE02	0.03	96.91	351.3	6
	GE03	0.003	94.72	362.4	13
	GE04	10^{-4}	90.43	367.0	44
	GE05	10^{-5}	84.45	385.4	111

G.5 Accuracy–Canvas-TPF Tradeoffs

For K_i committed blocks of size $B = 32$, define

$$\mathrm{TPF}_b^{\mathrm{canvas}} = \frac{1}{|\mathcal{C}_b|} \sum_{i \in \mathcal{C}_b} \frac{BK_i}{D_i}. \tag{13}$$

Unlike completion TPF, it includes final-block padding; it does not subtract edits or charge refinement/commit forwards. It is a refinement diagnostic, not wall TPS. The locked Quality sweep uses the same tasks at all fourteen settings, separately for HumanEval and GSM8K. At the highlighted HumanEval pair, both checkpoints pass 146/164 tasks; canvas TPF rises from 4.8395 to 13.9086 ($2.874\times$), while completion TPF rises from 4.3668 to 13.0771 ($2.995\times$). Seven tasks pass only each side; accuracy matching is empirical. Appendix J.2 gives comparisons at shared settings and curve validation.

H Additional Ablations

H.1 System Cost

At $n = 16$ rollouts, packing reduces actor-update time from 368 to 131 s and replay materialization from 55 to 16 s; cost balancing further reduces the update to 96 s. Across rollout counts, update cost drops from 0.60–0.93 to 0.24–0.25 s per 1k supervised tokens. Packing makes each replay row larger but reduces row count by about $5\times$, so per-row cost alone is misleading. Asynchronous host staging does not consistently improve per-unit cost; the large rollout gain requires GPU-resident handoff.

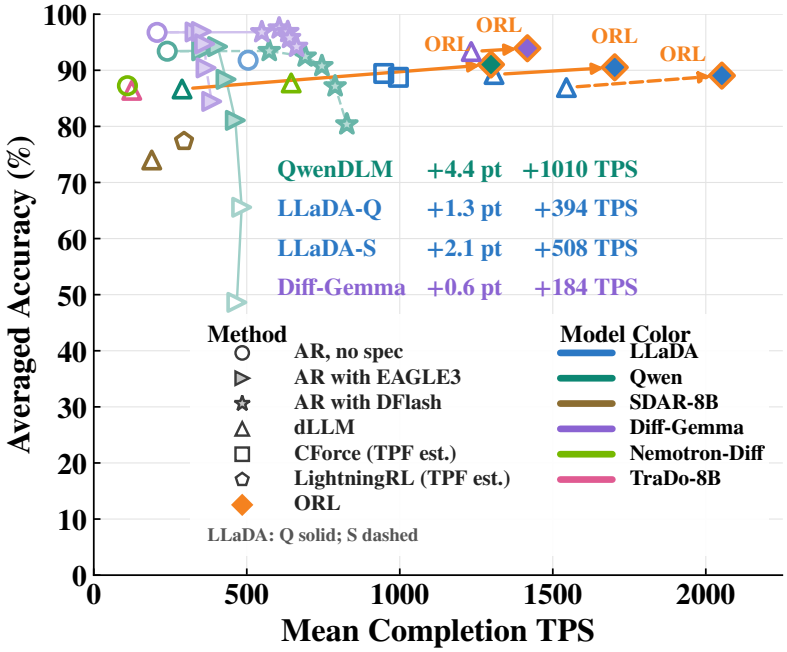


Figure 13: **Accuracy–throughput on GSM8K and HumanEval with additional baselines.** Both axes average the two benchmark-level metrics. Marker shape denotes the method and color the model family; orange arrows connect each Base DLM to its ORL model. LLaDA-Q and LLaDA-S denote Quality and Speed modes. TPS for CForce, LightningRL, and Nemotron-Diff is estimated as described above. Base and ORL DLM throughput is measured with DLMRT.

Paired handoff and memory. On identical trajectories at $n = 8, 16, 24$, GPU-resident handoff changes rollout time from 15.5/17.4/28.4 to 4.2/5.3/9.9 s and replay materialization from 10.7/14.3/34.1 to 2.0/3.2/5.9 s, with little actor-update change. This isolates transport from workload variation. At cap 2,048, the full system uses about 19 GB GPU and 113–116 GB host memory; baseline host use grows to 381/541/799 GB. At cap 8,192, the baseline exceeds the node's 944 GB host capacity. The plotted 359 GB system point disables GPU handoff because the full variant reaches its capture-arena limit; it does not demonstrate constant host memory at that length.

H.2 Residual Scaling and Block Size

Matched QwenDLM/GSM8K runs differ only in $\gamma = 8$ residual rescaling. Within their shared measurement window, excluding skipped updates, the median $100\|\Delta W\|_2/\|W\|_2$ for the residual MLP is 0.00142% with scaling and 0.04431% without (31.3×); backbone values are 0.00122%/0.00117% (0.96×). These are measured optimizer-dependent update ratios.

The MBPP block-size study complements Figure 6: ORL raises TPF from 8.6 to 13.0 at block size 32 and from 7.2 to 16.6 at 128.

H.3 Training Data and Compute

The data budget counts prompt and response tokens, including failed rollouts, without multiplying by training epochs. LLaDA training generates 5,000 responses: 0.74M prompt and 3.49M response tokens (4.2M total); 2.37M response tokens pass verification and enter the loss. Published budgets are about 40M/65M for d3LLM-LLaDA/Dream, 1B for Fast-dLLM-v2, and 54B for SDAR (Qian et al., 2026; Wu et al., 2026; Cheng et al., 2025). dParallel's 40M point is estimated from the same 92,428-trajectory data reused by d3LLM; counting only 256-token responses gives 23.7M (Chen et al., 2026c). We compare ORL's budget with post-training recipes, not AR-to-DLM conversion.

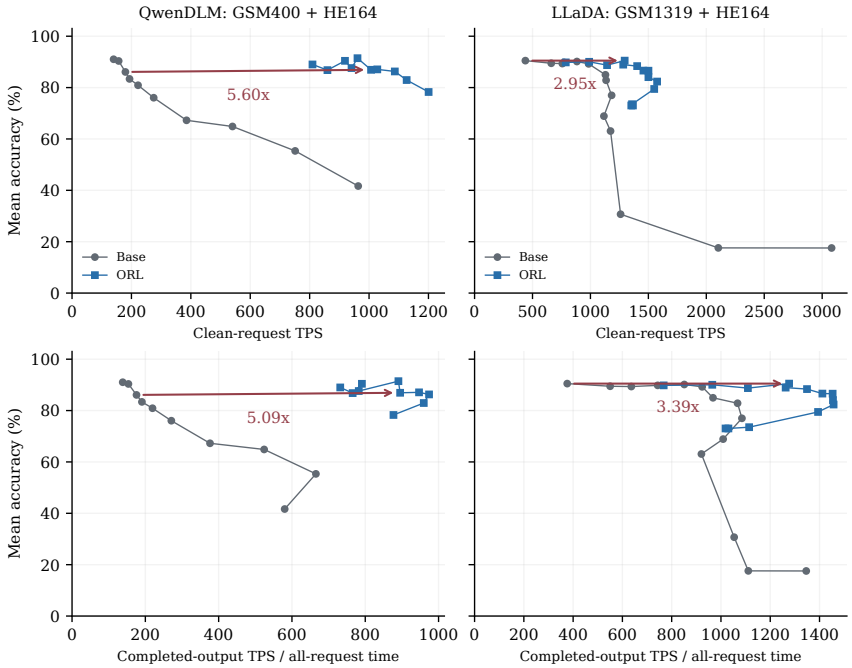


Figure 14: Common-task quality–throughput sweeps. Axes average GSM8K and HumanEval; all settings remain visible. Top: clean TPS; bottom: completed-output TPS; charging failed-request time. The highlighted Qwen pair (Base threshold 0.90, ORL entropy budget 0.80) gives $86.09/86.92\%$ mean accuracy and $5.60/5.09\times$ clean/completed TPS gains. The highlighted LLaDA pair (Base L14, ORL L11) gives 90.49% on both sides, with $2.95/3.39\times$ gains. Lines connect points in order of increasing threshold or entropy budget; matching refers to observed mean accuracy. All settings are measured with SGLang.

Table 14 counts every allocated H100 along the training chain leading to each reported checkpoint, including startup, model loading, rollout, verification, updates, and checkpoint writes. It excludes initializer production, checkpoint evaluation, other experimental runs, idle gaps, and attempts failing before their first checkpoint. Including the latter adds about 4.5 GPU-hours for QwenDLM and at most 8.5 for DiffusionGemma. When a job runs beyond the reported checkpoint, the next rollout timestamp bounds its endpoint to within one minute.

Table 14: Post-training compute for the reported checkpoints. Per-job hours follow chain order; GPU-hours count all allocated H100 80GB GPUs and are rounded.

Model	GPUs	Per job (h)	Total (h)	GPU-hours
QwenDLM	8	$5.00 + 0.66 + 1.79$	7.45	60
LLaDA-2.1-mini	8	$3.25 + 1.27$	4.52	36
DiffusionGemma	16	$8.22 + 9.55$	17.77	284

I Policy Dependence and Entropy

I.1 The Update and Stale Data

Let $w = (\theta, \phi)$ and let ν_w be the distribution of captured training records after verification, selection, and objective-specific weighting. At iteration u, ORL takes the frozen-data gradient

$$g_u = \nabla_w \mathbb{E}_{\xi \sim \nu_{w_u}} \ell(w; \xi)\big|_{w=w_u}. \tag{14}$$

Inputs, targets, and source entropies are held fixed for this differentiation. DiffusionGemma recomputes its source entropy during replay and then detaches it. Neither recipe differentiates through

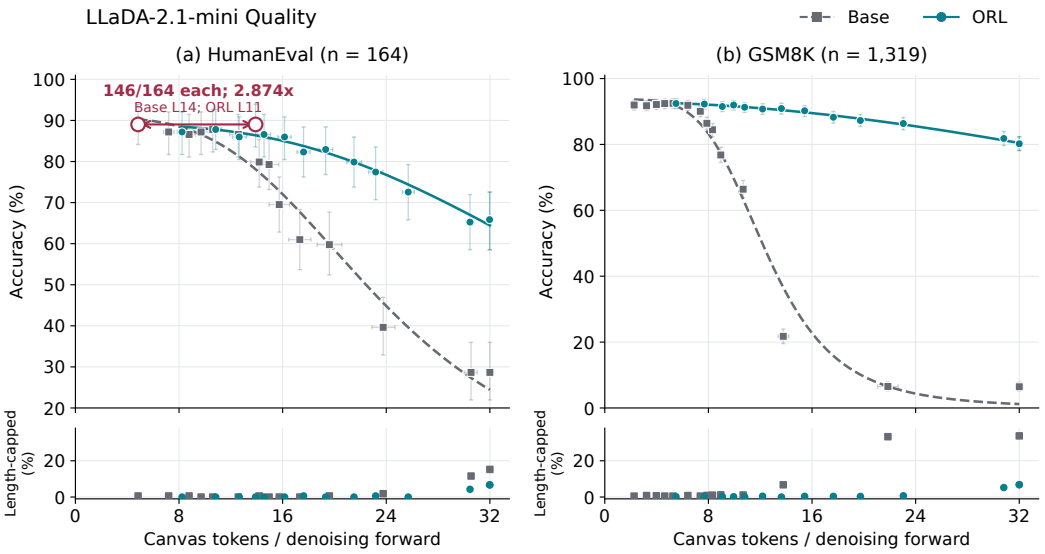


Figure 15: LLaDA Quality-mode Base versus ORL, with all operating points. Lines are independent log-logistic fits within observed ranges; bars are pointwise 95% percentile intervals from 2,000 paired task-vector resamples, conditional on the archived evaluation records. They quantify task-resampling uncertainty, excluding between-run variation and simultaneous coverage across operating points. The HumanEval arrow compares Base profile L14 (threshold 0.99) with ORL profile L11 (threshold 0.85); lower strips show length failures. Profiles are defined in Table 15.

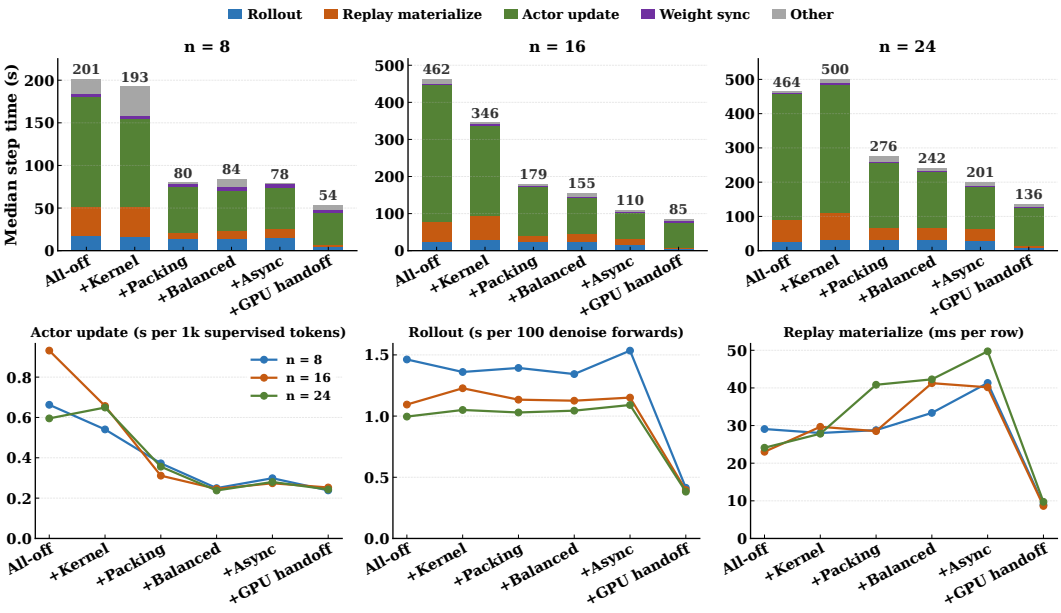
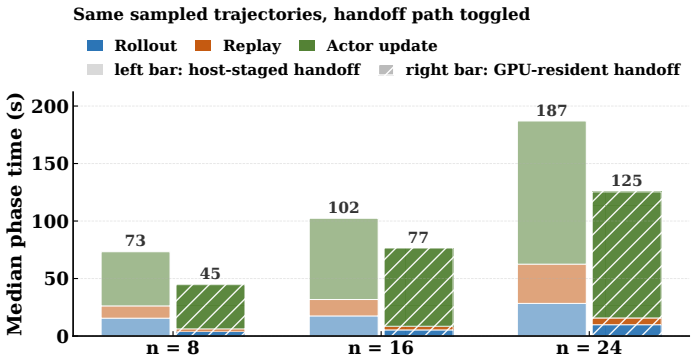
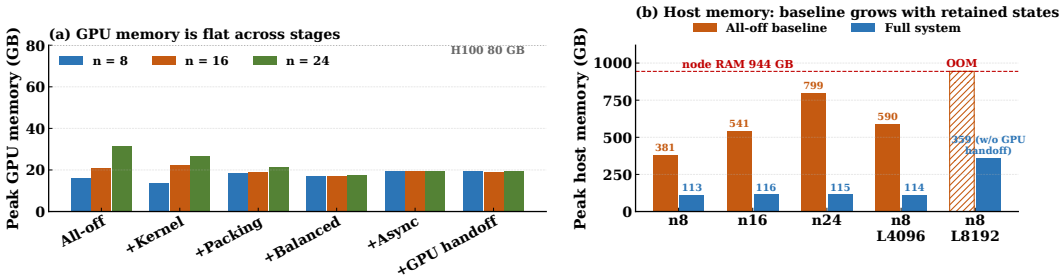


Figure 16: QwenDLM system ablation at response cap 2,048. Top: median phase times; bottom: median costs per 1k supervised tokens, 100 denoising forwards, and replay row. Medians use post-warmup steps; the main figure instead uses mean step times. Raw runs can differ in sampled workload.

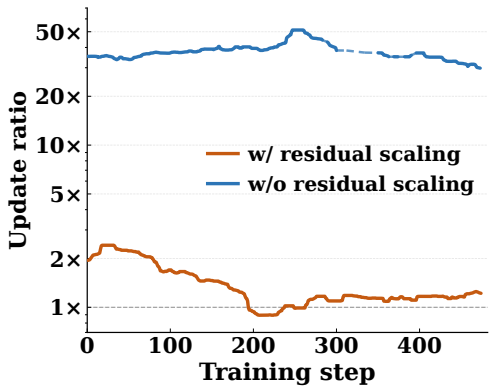


(a) Handoff paths on identical trajectories.

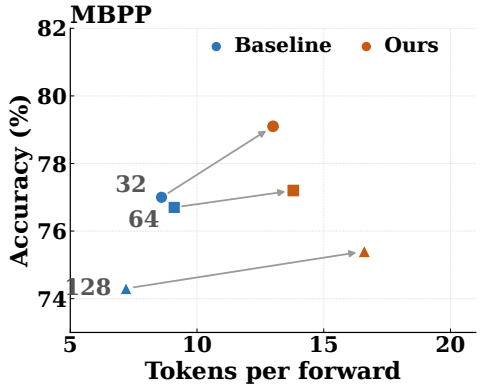


(b) GPU and host memory; 8,192-token caveat in text.

Figure 17: Transport and memory diagnostics. Paired handoff bars show median rollout/replay/update times; memory panels show peaks. All GPU configurations fit an 80 GB H100.



(a) Residual scaling controls relative updates.



(b) LLaDA training block size versus MBPP efficiency.

Figure 18: Residual-path stability and block-size sensitivity. Rescaling primarily changes residual, not backbone, update magnitudes.

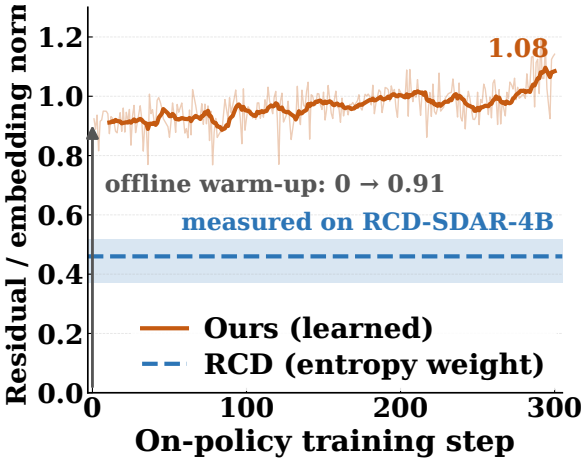


Figure 19: QwenDLM residual-to-embedding norm ratio, rising from 0.91 on the first on-policy rollouts to 1.08 during training. The dashed RCD-SDAR-4B reference is a separately measured entropy-weighted diagnostic. The offline-SFT RMS ratio of 1.06 at the same initializer uses a different population and statistic, so the values are not directly comparable. These traces describe residual use, not its causal benefit.

rollout generation or verification. In a differentiable common-density model, the full derivative of $\mathbb{E}_{\nu_w}\ell(w;\xi)$ would also contain $\mathbb{E}_{\nu_w}[\ell(w;\xi)\nabla_w \log \nu_w(\xi)]$. Thus verified CE is not itself a policy-gradient estimator of task reward.

Stale-gradient bound. At a fixed evaluation parameter w, assume $\|\nabla_w \ell(w;\xi)\|_2 \le G$ under both record distributions. Then

$$\|\mathbb{E}_{\nu_w}\nabla\ell(w;\xi) - \mathbb{E}_{\nu_v}\nabla\ell(w;\xi)\|_2 \le 2G\ \mathrm{TV}(\nu_w,\nu_v), \qquad \mathrm{TV}(P,Q) = \sup_A |P(A)-Q(A)|. \quad (15)$$

To prove this, project onto any unit vector and integrate the bounded scalar gradient against $\nu_w - \nu_v$, whose total variation norm is $2\,\mathrm{TV}(\nu_w,\nu_v)$. Taking the supremum proves the inequality. The gradient bound is an assumption, not automatic for CE. The result bounds possible mismatch, not harm: different data distributions can give the same gradient. If $\nu_w \ll \nu_v$, importance weighting can correct the expectation in principle, subject to support and variance. Deterministic continuous residuals may instead have disjoint supports. Fresh collection is therefore a design choice, not a proof that offline learning is impossible.

Verification and skipped updates. With one prompt, G independent rollouts, pass probability $p_w(c)$, and equal verified-response weights, the expected CE/hybrid gradient per attempted batch is $[1-(1-p_w(c))^G]g_w^+(c)$, where $g_w^+(c)$ is the conditional verified gradient. Indeed, a nonempty verified batch has expected mean $g_w^+(c)$, and its probability is $1-(1-p_w(c))^G$. Skipping all-wrong batches therefore reweights prompts. This calculation does not directly apply to gated AEC's weighted ratio, dependent rollouts, or nonlinear optimizer updates.

I.2 Entropy and Token Acceptance

Use unnormalized entropy $h(p) = -\sum_v p_v \log p_v$; the method uses $H = h/\log|\mathcal{V}|$. Suppose proposals are greedy and acceptance uses the same distribution at fixed threshold $0 < \tau < 1$. For n eligible positions,

$$\frac{\#\{i : \max_v p_{iv} \le \tau\}}{n} \le \min\left\{1, \frac{\bar{h}}{-\log\tau}\right\}. \quad (16)$$

Each failing position contributes at least $-\log\tau$ because $h(p_i) \ge -\log\max_v p_{iv}$; summing proves the bound. This concerns confidence, not correctness, and does not directly cover different confidence temperatures, sampled proposals, entropy budgets, or re-noising decoders.

If actual rollout means satisfy $H_{k+1} \le rH_k + e_k$ with $0 < r < 1$, induction gives $H_k \le r^k H_0 + \sum_{j<k} r^{k-1-j} e_j$. If all unresolved positions are eligible, all above-threshold greedy proposals commit, and positions never reopen, the remaining count obeys

$$n_K \le n_0 \prod_{k<K} \min\left\{1, \frac{H_k \log |\mathcal{V}|}{-\log \tau}\right\}. \tag{17}$$

The vocabulary factor converts normalized entropy to h. A right-hand side below one implies completion, not a correct answer. A finite AEC penalty on selected training pairs does not impose the assumed contraction on new rollouts.

Changing masks also changes the measurement. If $U' = U \setminus A$ with both A, U' nonempty, then

$$\bar{h}_{k+1}(U') - \bar{h}_k(U) = [\bar{h}_{k+1}(U') - \bar{h}_k(U')] + \frac{|A|}{|U|}[\bar{h}_k(U') - \bar{h}_k(A)]. \tag{18}$$

Removing confident positions can raise the remaining mean without worsening any prediction. Conversely, temperature sharpening lowers entropy: $dh(\mathrm{softmax}(\beta z))/d\beta = -\beta \operatorname{Var}_{p_\beta}(z) \le 0$, while leaving a wrong argmax wrong. These are reasons to measure quality separately.

I.3 Optimizer and Rescaling

At a fixed captured input and mask, $p = \mathrm{softmax}(z)$ gives

$$\Delta h = -\operatorname{Cov}_{v\sim p}(\log p_v, \Delta z_v) + O(\|\Delta z\|_2^2). \tag{19}$$

This follows from $\partial h/\partial z_v = -p_v(\log p_v + h)$. The actual Δz depends on the network Jacobian, all losses, clipping, and optimizer state; a policy-gradient covariance specialization need not describe CE/AEC/AdamW. Rollout-average changes additionally include state-distribution shift.

For a compensated projection $\widetilde{W} = \gamma W$ evaluated as $\widetilde{W}/\gamma$, the chain rule gives $\nabla_{\widetilde{W}} L = \nabla_W L/\gamma$. Equal-rate SGD reduces the effective W update by γ^2. Adam with consistently scaled moments and negligible epsilon reduces it by approximately γ, not γ^2; clipping, untransformed moments, and epsilon can change this relation. AdamW's relative decay is unchanged. Equation 6 proves forward-function preservation; stability and the measured update ratios remain empirical.

J Accuracy–TPF Curves

J.1 A Descriptive Model

For an operating point τ, write $S = \mathbf{1}\{\text{not length-terminated}\}$, Z for benchmark correctness, and $U = C/D$. The sweep estimates $a(\tau) = \mathbb{E}[SZ]$ and $t(\tau) = \mathbb{E}[U \mid S = 1]$, not $P(Z = 1 \mid U = t)$. In particular, $a = (1 - P(S = 0))P(Z = 1 \mid S = 1)$; collapse changes both axes. For the canvas diagnostic, replace C by BK throughout.

We use the log-logistic family

$$a(t) = \frac{A}{1 + (t/f_0)^\kappa}, \qquad 0 < A \le 1, \quad f_0, \kappa > 0. \tag{20}$$

One sufficient latent model assumes a solvable fraction A, sensitivity $\Lambda \sim \mathrm{Exp}(\eta)$ within that fraction, and retention $\exp(-\Lambda t^\kappa)$. Then

$$A \int_0^\infty e^{-\lambda t^\kappa} \eta e^{-\eta\lambda}\, d\lambda = \frac{A\eta}{\eta + t^\kappa}, \qquad f_0 = \eta^{1/\kappa}.$$

Exponential sensitivity and power exposure are assumptions, not consequences of residual decoding. A gamma sensitivity instead gives $A(1 + t^\kappa/\eta)^{-r}$. Neither identity explains why an aggregate, termination-conditioned TPF should equal the assumed exposure.

The curve has plateau A, half-plateau location f_0, and $da/d\log t|_{t=f_0} = -A\kappa/4$. At relative retention q,

$$t_q = f_0 \left(\frac{1-q}{q}\right)^{1/\kappa}, \qquad \frac{t_i(q)}{t_j(q)} = \frac{f_i}{f_j}\left(\frac{1-q}{q}\right)^{1/\kappa_i - 1/\kappa_j}. \tag{21}$$

Thus a quality-independent horizontal factor requires shared κ. At common absolute accuracy α, $t_m(\alpha) = f_m(A_m/\alpha - 1)^{1/\kappa_m}$. The ratio is constant on an open accuracy interval only if both plateaus and shapes match: differentiating its logarithm gives

$$\frac{d\log(t_R/t_B)}{d\alpha} = \frac{A_B}{\kappa_B\alpha(A_B-\alpha)} - \frac{A_R}{\kappa_R\alpha(A_R-\alpha)};$$

equating polynomial coefficients after cross-multiplication requires $\kappa_B = \kappa_R$ and $A_B = A_R$. Otherwise speed ratios must name their accuracy level. When each block requires a denoising forward, canvas TPF is at most B; $f_0 > B$ is an extrapolated parameter, not an attainable operating point. Even $t_{0.9}$ requires a measured-range check.

J.2 Locked LLaDA Sweep

All 56 Base/ORL points are reconstructed from 41,524 request records: fourteen Quality-mode profiles on HE164 and GSM1319 for each checkpoint. Task IDs match across profiles/checkpoints; accuracy, clean counts, and both TPF definitions reproduce saved aggregates within 10^{-10}. HumanEval uses base tests and GSM8K its original extractor.

Observed operating points. The HumanEval pair in Figure 15 has a $2.874\times$ canvas-TPF ratio at 146/164 correct on both sides. Recomputing on the 163 tasks clean for both gives $2.883\times$. For 20,000 paired task bootstraps, the canvas-ratio interval is $[2.737, 3.012]$ and the accuracy-difference interval is $[-4.268, 4.268]$ percentage points. These pointwise intervals condition on the archived evaluation records and the displayed pair; they quantify task-resampling uncertainty, without estimating between-run variability or accounting for comparisons across the full sweep. They do not establish population equivalence. On GSM8K, maximizing observed canvas TPF subject to empirical accuracy floors of 90%/92% gives ratios 2.093/1.461. The resulting operating points need not have identical accuracies.

Shared evaluation profiles. Table 15 compares Base and ORL under identical settings and retains every operating point, including quality losses. All profiles use block size 32, edit threshold 0.5, sixteen refinement steps, and confidence temperature 1.0, and differ only in the acceptance threshold. This comparison does not isolate AEC from the other training components.

Table 15: LLaDA Base–ORL comparisons across all fourteen shared profiles. Threshold: minimum top-token probability for accepting a position. Δa: ORL minus Base accuracy in percentage points. R: clean *completion*-TPF ratio, not padded canvas or wall TPS.

		HumanEval		GSM8K	
Profile	Threshold	Δa	R	Δa	R
L01	0.00	+37.20	1.048	+73.77	0.920
L02	0.05	+37.20	1.096	+73.69	1.380
L03	0.10	+25.61	1.344	+60.05	2.190
L04	0.20	+12.80	1.368	+19.94	2.111
L05	0.30	+16.46	1.392	+10.46	2.159
L06	0.40	+10.37	1.423	+3.79	2.077
L07	0.50	+3.66	1.352	+3.79	1.914
L08	0.60	+2.44	1.296	+0.83	1.812
L09	0.70	-0.61	1.346	-1.14	1.862
L10	0.80	-1.22	1.432	-1.21	2.011
L11	0.85	+1.83	1.500	-0.45	2.110
L12	0.90	-0.61	1.508	-0.61	2.235
L13	0.95	+0.61	1.574	+0.53	2.334
L14	0.99	-1.83	1.774	+0.45	2.379

At profile L11, HE accuracy is 143/164 versus 146/164 and completion TPF is 8.720 versus 13.077; GSM accuracy is 1219/1319 versus 1213/1319, with TPF 4.045 versus 8.535. Across 20,000 joint task-vector bootstraps (seed 20260925), a max-standardized-deviation band over 56 accuracy/log-ratio coordinates gives TPF-ratio intervals [1.385,1.623]/[2.059,2.163] and accuracy-difference intervals [-6.00,9.66]/[-2.74,1.83] for HE/GSM. These approximate familywise intervals cover the

displayed profiles and benchmarks conditional on the evaluated checkpoints and archived request records. They quantify task-resampling uncertainty without estimating variation across independent training runs or repeated evaluations.

Table 16: Conditional accuracy-prediction RMSE (percentage points), including endpoint folds. This holds out operating points, not tasks or seeds.

	One point out		Adjacent pair out	
	Log-logistic	Isotonic	Log-logistic	Isotonic
HE / Base	4.46	3.66	6.65	5.76
HE / ORL	1.69	1.42	2.39	1.31
GSM / Base	6.91	8.28	7.44	9.66
GSM / ORL	0.46	0.44	0.59	0.89

Held-out operating points. We fit each checkpoint/benchmark independently with a marginal binomial working loss, four initializations, and bounds $A \in [.001, 1]$, $f_0 \in [.1, 500]$, $\kappa \in [.1, 50]$. Each fold excludes one operating point or one adjacent pair in the ordered grid, predicting all fourteen points once. Both predictors receive held-out measured canvas TPF. The comparator is decreasing isotonic regression with linear interpolation, endpoint clamping, and pooled weights at duplicate coordinates.

The log-logistic fit is not uniformly better. In 400 paired task-vector bootstrap refits, 95% intervals for isotonic minus log-logistic adjacent-pair RMSE are [-4.18,3.00]/[-2.04,0.22] for HE Base/ORL and [1.10,3.30]/[-0.06,0.37] for GSM Base/ORL. Only GSM Base's individual interval excludes zero in favor of the curve; these intervals are exploratory and not simultaneous. Full-data Base/ORL shape estimates are 3.39/3.02 on HE and 4.64/2.02 on GSM, not evidence of a shared shape. ORL's half-plateau estimates 44.03/80.39 exceed the canvas bound 32. The curve is a compact empirical approximation, not an identified latent-difficulty law or proof of an AEC mechanism.